

PC 端 Web SDK Vue 接口说明文档 V4.0.6

文档修改记录

序号	版本号	修改内容	修改者	修改日期
1	v3.1.2	文档建立	周功成	2021/4/7
2	v3.1.4	更新 3.1.4 内容	石坤	2021/11/15
3	v3.1.8	- 兼容新机型 - 修复旋转后裁切异常 bug	石坤	2022/8/8
4	v3.1.8	更新接口封装	张彬	2023/6/7
5	v3.2.1	- 支持 K3 及 K3W 机型 - 增加 WIFI 相关接口	张彬	2023/10/12
6	v3.2.2	- 支持 M2 机型 - 增加 M2 相关说明	张彬	2023/10/30
7	v3.2.5	- 支持 B3S_P 机型 - 支持 B21S 机型 - 支持 B31 机型 - 更新图像库	张彬	2024/9/12
8	v4.0.3	- 支持 M3、K2、B21Pro 系列机型 - 完善错误码 - 新增绘制带 logo 二维码接口	张彬	2025/4/29

		提高 Websocket 通讯速度 5.Demo 支持黑标间隙纸		
9	v4.0.6	- 新增 closePrinter 接口 - 修复 Wifi 搜索接口 BUG - 修复历史遗留 WIFI 连接 BUG	张彬	2025/9/13
10	v4.0.6	修复文本换行模式BUG	张彬	2025/11/20

DEMO 目录结构

代码块

```
1  PC-SDK-VUE/
2  |— node-module/           # NPM 加载的项目依赖模块
3  |— public/                # 存放公共资源和项目主入口文件
4  |— src/                   # 项目核心文件夹：包括项目源码、静态资源
   等
5  |   |— asset/             # 存放静态资源的文件夹
6  |   |— router/            # 保存各类路由相关配置
7  |   |   |— index.js       # 路由配置
8  |   |— units/             # 存放接口文件的文件夹
9  |   |   |— printData/      # 存放打印数据的文件夹
10 |   |   |   |— Barcode.js  # 一维码打印及预览示例数据
11 |   |   |   |— Batch.js   # 批量打印及预览示例数据
12 |   |   |   |— Combination.js # 组合打印及预览示例数据
13 |   |   |   |— Graph.js   # 图形打印及预览示例数据
14 |   |   |   |— Img.js     # 图片打印及预览示例数据
15 |   |   |   |— Line.js    # 线条打印及预览示例数据
16 |   |   |   |— QrCode.js  # 二维码打印及预览示例数据
17 |   |   |   |— Text.js    # 文本打印及预览示例数据
18 |   |   |— Print.js        # 打印接口文件
19 |   |   |— Socket.js       # 打印服务连接文件
20 |   |   |— PrintElementFactory.js # 绘制工厂策略类
21 |   |— views/              # 存储页面文件
22 |   |   |— HomeView.vue    # 打印示例文件
23 |   |— App.vue             # 页面入口文件
24 |   |— balel.config.js     # 全局配置文件
```

```
25 |   |— jsconfig.json           # 指定了根文件以及JavaScript语言服务提供
   |   的功能选项
26 |   |— package-lock.json      # 项目版本管理使用的文件
27 |   |— package.json          # 项目的基本配置文件
28 |— vue.config.js             # 配置文件
```

产品目的

JCAPI 接口为调用者提供易用的方法完成标签绘图、打印操作。本接口中提供了标贴的绘制方法，包括：文字、一维码、二维码，图形、线条、图像绘制，同时还能进行绘制对象的旋转，调用者还可以调用方法获得绘制完成的标签图片用于标签预览，打印。方便用户在二次开发中调用接口，缩短开发周期，加快开发

打印机支持

支持打印机型号
B1
B203
B21 /B21_Pro/B21S
B3S / B3S_P
B31
B4
B11
K2
K3/K3W
B50/B50W
B32/Z401/B32R
M2
M3

准备工作

- 安装精臣打印服务（jcPrinterSdk.exe）
 - 前置：关闭杀毒软件（如 360，易误报）
 - 关键：**必须默认路径安装（C 盘）**
 - 注意：勿禁用服务开机启动
- 安装对应机型驱动

机型系列	系统要求
B50/B11	Win7/10/11 均需装驱动
其他机型	Win10/11 无需装，仅 Win7 需装

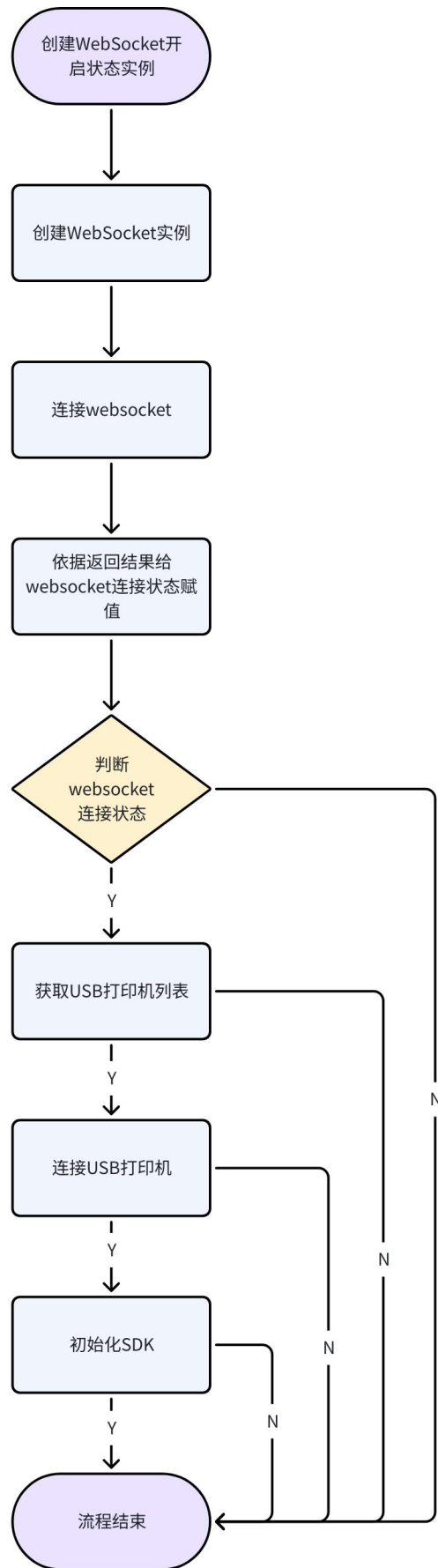
- 设备连接（2 种方式，不支持蓝牙）
 - USB 连接
 - 系统：仅支持 Windows
 - 驱动：可能需装（参考第 2 步）
 - 特别：****不支持驱动打印****（已用驱动打印需下载专用驱动）
 - WIFI 连接
 - 机型：**仅支持 K3W 机型**
 - 系统：仅支持 Windows
 - 驱动：无需安装

一、初始化及打印调用流程、打印流程

1.1 初始化流程

1.1.1 USB 打印初始化流程

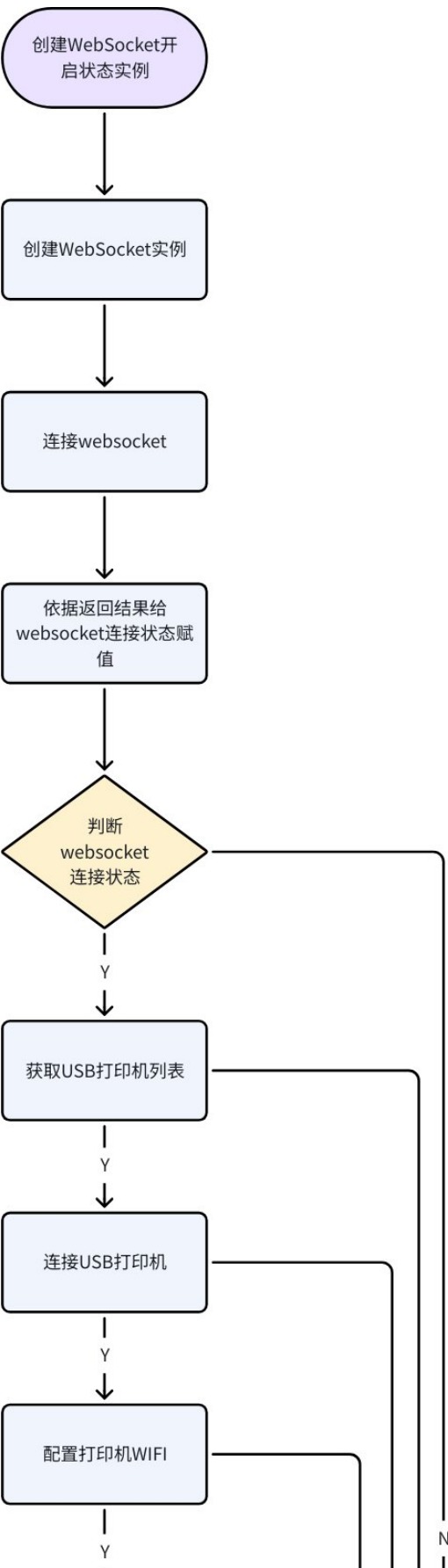
- WebSocket 建议页面加载时进行初始化，在 WebSocket 初始成功后回调中进行获取打印机、选择打印机、初始化 SDK 等操作
- 因为所有接口均为异步操作，调用下一接口需要验证当前接口结果后再执行下一接口
- 记录打印机列表获取状态、连接状态、初始化状态，打印机需要检查对应的状态

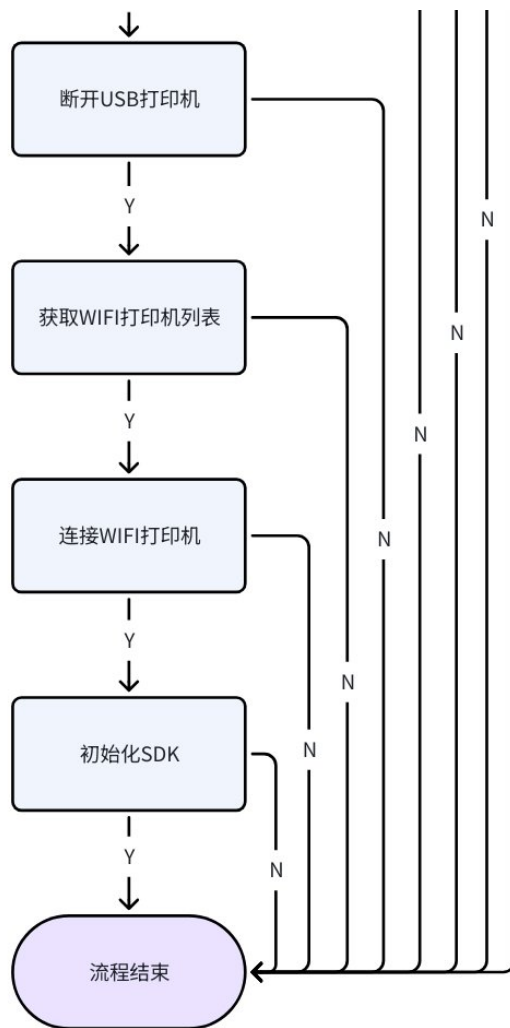


1.1.2 WIFI 打印初始化流程

- WebSocket 建议页面加载时进行初始化，在 WebSocket 初始成功后回调中进行获取打印机、选择打印机、初始化 SDK 等操作

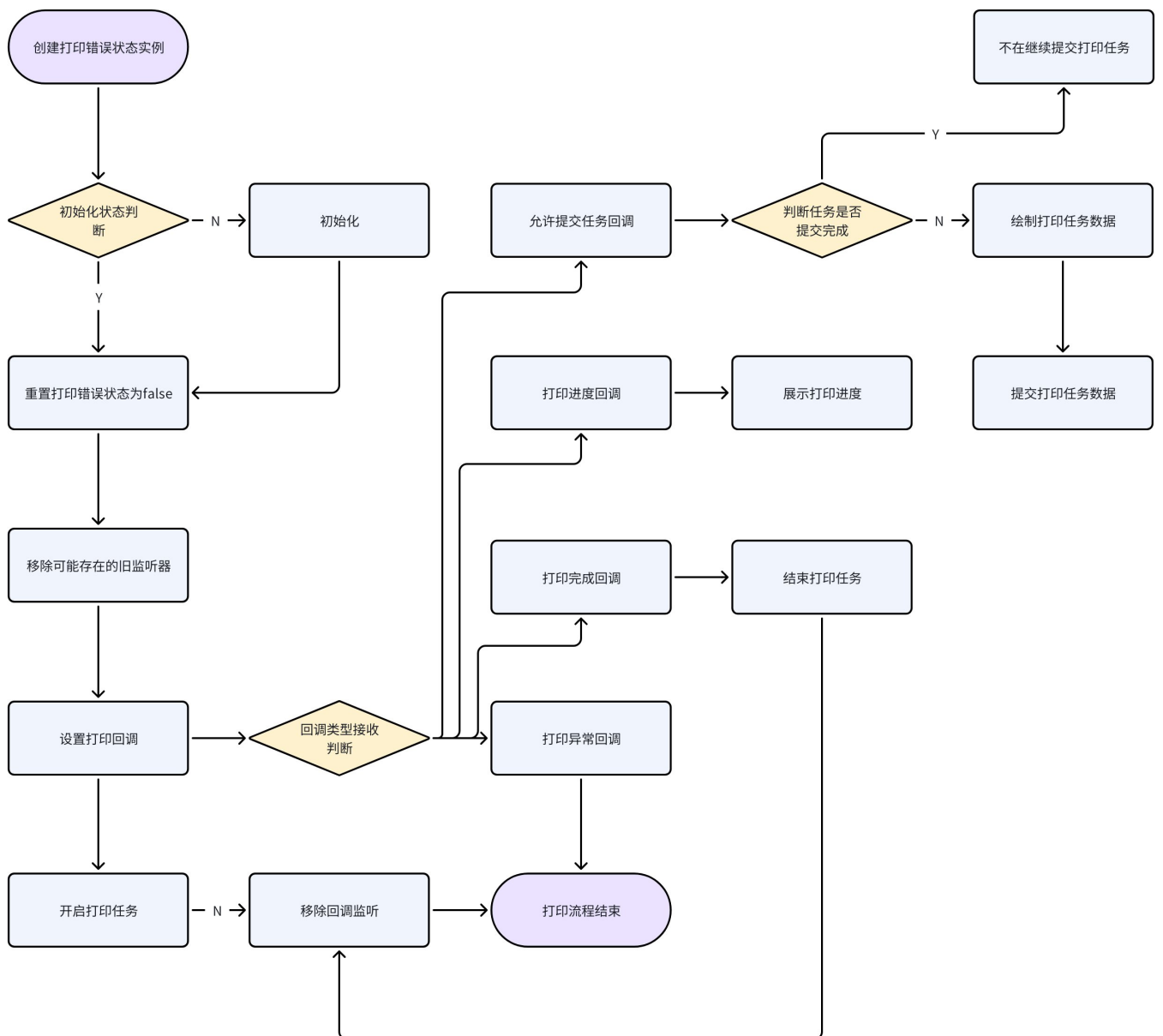
- 因为所有接口均为异步操作，调用下一接口需要验证当前接口结果后再执行下一接口
- 记录打印机列表获取状态、连接状态、初始化状态，打印机需要检查对应的状态
- 打印机 WIFI 配置成功后，后续直接搜索连接，无需多次进行配置（省略 USB 打印机获取、打印机连接、打印机网络配置）





1.2 打印流程

- 打印前建议判断 WebSocket 是否初始化成功、SDK 是否初始化成功（包含初始化 SDK，获取打印机、选择打印机三个流程）
- 因为所有接口均为异步操作，除 WebSocket 初始化调用是在单独的回调接口中判断是否初始化成功外，其他接口应通过添加 await 关键字调用方法后，等待方法返回结果，解析返回的结果数据后进行判断再进行下一接口调用
- 打印回调监听会有多种回调，包含异常取消、页码回调，可参考流程图及 DEMO 进行处理



二、页面初始化相关接口

2.1 初始化打印服务及接口实例（包含打印机状态回调）

代码块

```

1  export default class Socket {
2    /**
3     * 打开 WebSocket 连接并返回一个解析为 WebSocket 实例的 Promise。
4     *
5     * @param {function} openChange - WebSocket 连接打开时要调用的回调函数。
6     * @param {function} onMessageCallback - 接收到消息时要调用的回调函数。
7     * @return {Promise} 一个解析为 WebSocket 实例的 Promise。

```



```

8      */
9      open(openChange, onMessageCallback)
10  }

```

代码块

```

1  // 创建socket实例
2  const socketData = new Socket();
3  socketData.open(
4      (openBool) => {
5          console.log(openBool, "openBool");
6          this.printSocketOpen = openBool;
7      },
8      (msg) => {
9          if (msg.resultAck.callback != undefined) {
10             const callbackName = msg.resultAck.callback.name;
11             const msgInfo = msg.resultAck.info;
12             if (callbackName == "onCoverStatusChange") {
13                 //盒盖状态: 0-闭合、1-打开
14                 console.log("盒盖状态", msgInfo.capStatus);
15             } else if (callbackName == "onElectricityChange") {
16                 // "power" : 0-4, // 电池电量等级 (共5档)
17                 console.log("电池电量等级", msgInfo.power);
18             }
19         }
20     }
21 );

```

2.2 初始化 SDK initSdk

代码块

```

1  export default class NMPrintSocket {
2      /**
3       * 初始化SDK, 在打印服务连接成功后调用此接口。
4       * 在调用SDK的绘制接口之前, 必须先调用此接口。
5       *
6       * @param {object} json - 包含必要参数的JSON对象, 格式如下:
7       * {
8       *     "fontDir": string, //字体文件目录, 默认为"", 暂不生效
9       * }
10      *
11      * @return {Promise} 返回一个 Promise, 解析为初始化SDK的结果
12      */

```

```
13     initSdk(json)
14 }
```

代码块

```
1  //初始化SDK参数JSON
2  {
3      "fontDir": ""
4  }
5  //初始化成功返回JSON
6  {
7      "apiName": "initSdk",
8      "resultAck": {
9          "errorCode": 0,
10         "info": "initSdkApi ok!",
11         "result": 0
12     }
13 }
14
15 // 创建打印实例,此实例只需创建一次
16 this.nMPrintSocket = new NMPrintSocket(socketData);
17 //进行初始化
18 async init() {
19     if (!this.printSocketOpen) return alert("打印服务未开启");
20     //初始化数据
21     try {
22         const res = await this.nMPrintSocket.initSdk({ fontDir: "" });
23         if (res.resultAck.result == 0) {
24             console.log("初始化成功");
25             this.initBool = true;
26         } else {
27             console.log("初始化失败");
28             this.initBool = false;
29         }
30     } catch (err) {
31         console.error(err);
32     }
33 }
```

2.3 获取 USB 打印机列表 getAllPrinters

代码块

```
1  export default class NMPrintSocket {
2      /**
3       * 获取所有当前PC上连接的精臣打印机
```

```

4      *
5      * @return {Promise} 返回一个Promise, 解析为打印机列表。
6      *
7      * @description
8      * 需要在打印服务连接成功后调用此函数。
9      */
10     getAllPrinters()
11 }

```

代码块

```

1  //返回结果
2  {
3      "apiName": "getAllPrinters",
4      "resultAck": {
5          "errorCode": 0,
6          "info": "{\\\"e623012991\\\":\\\"31\\\"}", //打印机名称及类型
7          "result": "true"
8      }
9  }
10
11 // 创建打印实例,此实例只需创建一次
12 this.nMPrintSocket = new NMPrintSocket(socketData);
13 //调用流程
14 async getPrinters() {
15     if (!this.printSocketOpen) {
16         return alert("打印服务未开启");
17     }
18     console.log("开始获取打印机");
19     try {
20         const allPrintersRes = await this.nMPrintSocket.getAllPrinters();
21         console.log(allPrintersRes, "allPrintersRes");
22         if (allPrintersRes.resultAck.errorCode === 0) {
23             const allPrinters = JSON.parse(allPrintersRes.resultAck.info);
24             this.printers = { ...allPrinters };
25             this.selectPrinter = Object.keys(this.printers)[0];
26             console.log("printers", this.printers);
27         } else {
28             alert("没有在线的打印机");
29         }
30     } catch (err) {
31         console.error(err);
32     }
33 }

```

2.4 获取 WIFI 连接的打印机列表 scanWifiPrinter

代码块

```
1  export default class NMPrintSocket {
2    /**
3     * 搜索Wifi打印机
4     *
5     *
6     * @return {Promise} 返回一个 Promise, 解析为打印机Wifi配置信息
7     *
8     * @description
9     * 需要在打印服务连接成功后调用此函数。
10   */
11   scanWifiPrinter()
12 }
```

代码块

```
1  //返回结果
2  {
3      "apiName": "scanWifiPrinter",
4      "resultAck": {
5          "errorCode": 0,
6          "info": "[{
7              "deviceName": "K3W-E828013369",
8              "IP": "192.168.1.10",
9              "tcpPort": "9200",
10             "avaliableClient": "0"
11         }]",
12     "result": "true"
13     }
14 }
15
16
17 // 创建打印实例,此实例只需创建一次
18 this.nMPrintSocket = new NMPrintSocket(socketData);
19 //调用流程
20 async scanWifiPrinters() {
21     const allPrintersRes = await this.nMPrintSocket.scanWifiPrinter();
22     console.log("allPrintersRes", allPrintersRes);
23     const errorCode = allPrintersRes.resultAck.errorCode;
24     //处理搜索结果
25     if (errorCode === 0) {
26         const allPrinters = allPrintersRes.resultAck.info;
27         this.wifiPrinters = {};
```

```

28     allPrinters.forEach((item) => {
29         this.wifiPrinters[item.deviceName] = item.tcpPort;
30     });
31     console.log("wifiPrinters", this.wifiPrinters);
32
33     this.wifiSelectPrinter = Object.keys(this.wifiPrinters)[0];
34     console.log("wifiSelectPrinter", this.wifiSelectPrinter);
35 } else {
36     alert("没有在线的打印机");
37 }
38 },

```

2.5 连接 USB 打印机 selectPrinter

代码块

```

1  export default class NMPrintSocket {
2      /**
3       * 选择并打开需要使用的打印机名称，及端口号
4       *
5       * @param {string} printerName - 打印机名称。
6       * @param {number} port - 连接端口。
7       * @return {Promise} 返回一个Promise，解析为连接结果
8       *
9       * @description
10      * 需要在打印服务连接成功后调用此函数，建议在getAllPrinters调用成功后调用该接口，以保证传入的打印机名称和端口的打印机状态正常。。
11      */
12      selectPrinter(printerName, port)
13  }

```

代码块

```

1  //返回数据示例
2  {
3      "apiName": "selectPrinter",
4      "resultAck": {
5          "callback": {
6              "name": "onConnectSuccess",
7              "printerName": "e623012991"
8          },
9          "errorCode": 0,
10         "info": "select printer ok!",
11         "result": true
12     }
13 }

```

```

14
15 // 创建打印实例,此实例只需创建一次
16 this.nMPrintSocket = new NMPrintSocket(socketData);
17 //调用流程
18 async selectOnLinePrinter() {
19     if (!this.printSocketOpen) {
20         return alert("打印服务未开启");
21     }
22     console.log("开始连接打印机");
23     try {
24         const res = await this.nMPrintSocket.selectPrinter("e623012991",31));
25         console.log("选择打印机", res);
26         if (res.resultAck.result) {
27             console.log("连接成功");
28             this.onlineBool = true;
29         } else {
30             console.log("连接失败");
31             this.onlineBool = false;
32             alert("连接失败");
33         }
34     } catch (err) {
35         console.error(err);
36     }
37 }

```

2.6 连接 WIFI 打印机列表中的打印机 connectWifiPrinter

代码块

```

1  export default class NMPrintSocket {
2      /**
3       * 发送消息以选择打印机。
4       *
5       * @param {string} printerName - 打印机名称。
6       * @param {number} tcpPort - 端口号。
7       * @return {Promise} 返回连接结果
8       *
9       * @description
10      * 需要在打印服务连接成功后调用此函数，建议在scanWifiPrinter调用成功的回调接口中调用该
      接口，保证传入的打印机名称和端口的打印机状态正常。
11      * 注意：此函数仅能连接 WIFI 打印机列表中的打印机。
12      */
13      connectWifiPrinter(printerName, tcpPort)
14  }

```

14. 示例返回成功数据

```

2  {
3      "apiName": "selectPrinter",
4      "resultAck": {
5          "callback": {
6              "name": "onConnectSuccess",
7              "printerName": "e623012991"
8          },
9          "errorCode": 0,
10         "info": "select printer ok!",
11         "result": true
12     }
13 }
14 //示例返回失败数据
15 {
16     "apiName": "connectWifiPrinter",
17     "resultAck": {
18         "callback": {
19             "name": "onDisConnect",
20             "printerName": "K3_W-F612010061"
21         },
22         "errorCode": 0,
23         "info": "success",
24         "result": false
25     }
26 }
27
28 // 创建打印实例,此实例只需创建一次
29 this.nMPrintSocket = new NMPrintSocket(socketData);
30 //调用流程
31 async selectOnLineWifiPrinter() {
32     if (!this.printSocketOpen) {
33         return alert("打印服务未开启");
34     }
35     try {
36         const wifiConnectRes = await this.nMPrintSocket.connectWifiPrinter(
37             this.wifiSelectPrinter,
38             parseInt(this.wifiPrinters[this.wifiSelectPrinter])
39         );
40         //此版文报存在问题, errorCode连接成功与连接失败一致, 暂时先用result判断
41         const result = JSON.parse(wifiConnectRes.resultAck.result);
42         if (result) {
43             console.log("连接成功");
44             this.onlineWifiBool = true;
45             this.onlineUsbBool = false;
46         } else {
47             console.log("连接失败");

```

```

48         this.onlineWifiBool = false;
49         alert("连接失败");
50     }
51     console.log("wifiConnectRes", wifiConnectRes);
52 } catch (err) {
53     console.error(err);
54 }
55 },

```

2.7 断开打印机连接 closePrinter

代码块

```

1  export default class NMPrintSocket {
2      /**
3       * 断开打印机连接。
4       *
5       * @return {Promise} 返回一个Promise，解析为关闭结果
6       */
7      closePrinter()
8  }

```

2.8 配置打印机的 WIFI 信息 configurationWifi

代码块

```

1  export default class NMPrintSocket {
2      /**
3       * 配置打印机的Wifi网络
4       *
5       * @param {string} wifiName - wifi网络的名称。
6       * @param {string} wifiPassword - wifi网络的密码。
7       * @return {Promise} 返回一个 Promise，解析为打印机Wifi配置结果
8       *
9       * @description
10      * 注意:仅支持2.4G频段网络，且需要在连接成功后配置。首次配置建议在USB连接成功后配置
11      */
12      configurationWifi(wifiName, wifiPassword)
13  }

```

代码块

```

1  //示例返回数据
2  {
3      "apiName": "configurationWifi",

```



```

4     "resultAck":{
5         "errorCode":0,
6         "info":"configuration wifi printer ok!",
7         "result":true
8     }
9 }
10
11 // 创建打印实例,此实例只需创建一次
12 this.nMPrintSocket = new NMPrintSocket(socketData);
13 //调用流程
14 async setWifiConfiguration() {
15     if (!this.printSocketOpen) {
16         return alert("打印服务未开启");
17     }
18
19     try {
20         if (this.wifiName.trim() !== "") {
21             const wifiConfigurationResult =
22                 await this.nMPrintSocket.configurationWifi(
23                     this.wifiName.trim(),
24                     this.wifiPassword.trim()
25                 );
26
27             console.log("wifiConfigurationResult", wifiConfigurationResult);
28
29             const errorCode = JSON.parse(
30                 wifiConfigurationResult.resultAck.errorCode
31             );
32
33             console.log("errorCode", errorCode);
34
35             if (errorCode === 0) {
36                 return alert(
37                     "网络配置成功，请断开USB线缆后使用WIFI搜索连接打印机（PC需要和打印机在同一网
38                     络） "
39                 );
40             } else {
41                 return alert("网络配置失败");
42             }
43         } else {
44             return alert("wifi名称不得为空");
45         }
46     } catch (err) {
47         console.error(err);
48     },

```

2.9 获取打印机的 WIFI 相关配置 getWifiConfiguration

代码块

```
1  export default class NMPrintSocket {
2      /**
3       * 获取打印机的wifi配置。
4       *
5       * @return {Promise} 返回一个 Promise, 解析为打印机Wifi配置信息
6       */
7      getWifiConfiguration()
8  }
```

代码块

```
1  //示例返回成功数据
2  {
3      "apiName":"getWifiConfiguration",
4      "resultAck":{
5          "errorCode":0,
6          "info":{"
7              \n\t\"wifiName\" : \"Test\\\"\\n
8              }\n",
9          "result":{"
10             \n\t\"wifiName\" : \"Test\\\"\\n
11             }\n"
12         }
13     }
14 //示例返回失败数据
15 {
16     "apiName":"getWifiConfiguration",
17     "resultAck":{
18         "errorCode":23,
19         "info":"select printer connect first!",
20         "result":false
21     }
22 }
23
24 // 创建打印实例,此实例只需创建一次
25 this.nMPrintSocket = new NMPrintSocket(socketData);
26 //调用流程
27 async getWifiConfigurationInfo() {
28     if (!this.printSocketOpen) {
29         return alert("打印服务未开启");
30     }
31     try {
32         const wifiInfo = await this.nMPrintSocket.getWifiConfiguration();
```

```

33     const errorCode = JSON.parse(wifiInfo.resultAck.errorCode);
34
35     if (errorCode === 0) {
36         const info = JSON.parse(wifiInfo.resultAck.info);
37         console.log("wifiInfo", info);
38         alert("wifiInfo:" + info);
39     } else {
40         alert("wifiInfo:获取失败");
41     }
42 } catch (err) {
43     console.error(err);
44 }
45 },

```

三、绘制打印数据相关接口

3.1 创建画板 InitDrawingBoard

代码块

```

1  export default class NMPrintSocket {
2      /**
3       * 初始化绘制画板
4       *
5       * @param {Object} json - 包含初始化绘制画板所需数据的JSON对象。格式如下：
6       * {
7       *     "width": number, // 画板的宽度，单位为mm
8       *     "height": number, // 画板的高度，单位为mm
9       *     "rotate": number, // 画板的旋转角度，仅支持0、90、180、270
10      *     "path": string, // 字体文件的路径，默认为"", 暂不生效
11      *     "verticalShift": number, // 垂直偏移量，暂不生效
12      *     "HorizontalShift": number // 水平偏移量，暂不生效
13      * }
14      * @return {Promise} 返回一个 Promise，解析为初始化绘制画板的结果
15      *
16      * @description
17      * 增加接口说明：
18      * 1. 在调用绘制接口之前，必须先初始化SDK。
19      * 2. 绘制元素前，必须先初始化画板，否则会引起崩溃！
20      * 3. 初始化画板时会清空画板上次绘制的内容！
21      */
22      InitDrawingBoard(json)
23  }

```

代码块

```
1  {
2      "apiName": "InitDrawingBoard",
3      "resultAck": {
4          "errorCode": 0,
5          "info": "init draw board success!",
6          "result": 0
7      }
8  }
9
10 // 创建打印实例,此实例只需创建一次
11 this.nMPrintSocket = new NMPrintSocket(socketData);
12 // 调用流程
13 async InitDrawingBoard(){
14     const InitDrawingBoardParam={
15         "width":48,
16         "height":30,
17         "rotate":0,
18         "path":"ZT001.ttf",
19         "verticalShift":0,
20         "HorizontalShift":0};
21     //设置画布尺寸
22     try {
23         const res = await this.nMPrintSocket.InitDrawingBoard(
24             InitDrawingBoardParam
25         );
26         if (res.resultAck.result !== 0) {
27             return;
28         }
29         // //进行下一步操作,绘制元素
30     } catch (err) {
31         console.error(err);
32     }
33 }
```

3.2 绘制文本 DrawLableText

代码块

```
1  export default class NMPrintSocket {
2      /**
3       * 绘制标签文本。
4       * @param {object} json - 包含标签文本信息的JSON对象。
5       *   JSON格式要求如下:
```

```

6      *      - x: x轴坐标, 单位mm
7      *      - y: y轴坐标, 单位mm
8      *      - height: 文本高度, 单位mm
9      *      - width: 文本宽度, 单位mm
10     *      - value: 文本内容
11     *      - fontFamily: 字体名称, 暂不生效, 使用默认字体鸿蒙字体
12     *      - rotate: 旋转角度, 0, 90, 180, 270
13     *      - fontSize: 字号, 单位mm
14     *      - textAlignHorizonral: 水平对齐方式: 0:左对齐 1:居中对齐 2:右对齐
15     *      - textAlignVertical: 垂直对齐方式: 0:顶对齐 1:垂直居中 2:底对齐
16     *      - letterSpacing: 字母之间的标准间隔, 单位mm
17     *      - lineSpacing: 行间距 (倍距), 默认1
18     *      - lineMode: 1:宽高固定, 内容大小自适应, 预设宽高过大时字号放大, 预设宽高过小时字
号缩小,
19     *          保证内容占据满预设宽高 (字号/字符间距/行间距 按比例缩放)
20     *          2:宽度固定, 高度自适应
21     *          4:宽高固定,超出内容直裁切
22     *          6:宽高固定, 内容超过预设的文本宽高自动缩放
23     *          建议设置为6
24     *      - fontStyle: 字体样式[加粗, 斜体, 下划线, 删除下划线 (预留)]
25     *
26     * @return {Promise} 返回一个 Promise, 解析为绘制标签文本的结果
27     * @description 绘制标签文本前必须先初始化画板
28     */
29     DrawLableText(json)
30 }

```

代码块

```

1  //返回数据示例
2  {
3      "apiName": "DrawLableText",
4      "resultAck": {
5          "errorCode": 0,
6          "info": "draw bar code success!",//此处返回信息有误, 下个版本修复
7          "result": 0
8      }
9  }
10
11
12 // 创建打印实例,此实例只需创建一次
13 this.nMPrintSocket = new NMPrintSocket(socketData);
14 // 调用流程
15 async DrawLableText(){
16     const DrawLableTextParam = {
17         "x": 20.0,

```

```

18     "y": 10.0,
19     "height": 10,
20     "width": 50,
21     "value": "精臣SDK",
22     "fontFamily": "宋体",
23     "rotate": 0,
24     "fontSize": 4.0,
25     "textAlignHorizontal": 0,
26     "textAlignVertical": 0,
27     "letterSpacing": 1.0,
28     "lineSpacing": 1.0,
29     "lineMode": 0,
30     "fontStyle": [false, false, false, false]
31 }
32
33     const res = await this.nMPrintSocket.DrawLableText(DrawLableTextParam);
34     if (res.resultAck.result !== 0) {
35         return;
36     }
37     //进行下一步操作,继续绘制或提交
38 }

```

3.3 一维码绘制 DrawLableBarcode

代码块

```

1  export default class NMPrintSocket {
2      /**
3       * 绘制一维码条形码。
4       *
5       * @param {Object} json - 包含一维码条形码信息的JSON对象。格式如下:
6       * {
7       *     "x": number, // x轴坐标, 单位mm
8       *     "y": number, // y轴坐标, 单位mm
9       *     "height": number, // 一维码宽度, 单位mm
10      *     "width": number, // 一维码高度, 单位mm (包含文本高度)
11      *     "value": string, // 一维码内容
12      *     "codeType": number, // 条码类型:
13      *         // 20: CODE128
14      *         // 21: UPC-A
15      *         // 22: UPC-E
16      *         // 23: EAN8
17      *         // 24: EAN13
18      *         // 25: CODE93
19      *         // 26: CODE39
20      *         // 27: CODEBAR

```

```

21      *          // 28: ITF25
22      *      "rotate": number, // 旋转角度, 0, 90, 180, 270
23      *      "fontSize": number, // 文本字号, 单位mm, 字号为0则文本不显示
24      *      "textHeight": number, // 文本高度, 单位mm, 高度为0则文本不显示
25      *      "textPosition": number // 一维码文字识别码显示位置:
26      *          // 0: 下方显示
27      *          // 1: 上方显示
28      *          // 2: 不显示
29      *  }
30      *
31      * @return {Promise} 返回一个 Promise, 解析为绘制一维码条形码的结果
32      *
33      * @description
34      * 1. 绘制元素前, 必须先初始化画板
35      */
36      DrawLableBarCode(json)
37  }

```

代码块

```

1  //返回数据示例
2  {
3      "apiName": "DrawLableBarCode",
4      "resultAck": {
5          "errorCode": 0,
6          "info": "draw bar code success!",
7          "result": 0
8      }
9  }
10
11 // 创建打印实例, 此实例只需创建一次
12 this.nMPrintSocket = new NMPrintSocket(socketData);
13 // 调用流程
14 async DrawLableBarCode(){
15     const DrawLableBarCodeParam = {
16         "x": 20.0,
17         "y": 10.0,
18         "height": 10,
19         "width": 50,
20         "value": '12345678',
21         "codeType": 20,
22         "rotate": 0,
23         "fontSize": 4.0,
24         "textHeight": 0,
25         "textPosition": 0,
26     }

```

```

27
28         const res = await
this.nMPrintSocket.DrawLableBarCode(DrawLableBarCodeParam);
29     if (res.resultAck.result !== 0) {
30         return;
31     }
32     //进行下一步操作,继续绘制或提交
33 }

```

3.4.1 二维码绘制 DrawLableQrCode

代码块

```

1  export default class NMPrintSocket {
2      /**
3       * 绘制二维码。
4       *
5       * @param {Object} json - 包含二维码信息的JSON对象。格式如下:
6       * {
7       *     "x": number, // x轴坐标, 单位mm
8       *     "y": number, // y轴坐标, 单位mm
9       *     "height": number, // 二维码高度, 默认宽高一致
10      *     "width": number, // 二维码宽度, 单位mm
11      *     "value": string, // 二维码内容
12      *     "codeType": number, // 条码类型:
13      *                          // 31: QR_CODE
14      *                          // 32: PDF417
15      *                          // 33: DATA_MATRIX
16      *                          // 34: AZTEC
17      *     "rotate": number, // 旋转角度, 仅支持0、90、180、270
18      * }
19      *
20      * @return {Promise} 返回一个 Promise, 解析为绘制二维码的结果
21      *
22      * @description
23      * 1. 绘制元素前, 必须先初始化画板
24      */
25      DrawLableQrCode(json)
26  }

```

代码块

```

1  //返回数据示例
2  {
3      "apiName": "DrawLableQrCode",
4      "resultAck": {

```



```

5         "errorCode": 0,
6         "info": "draw qr code success!",
7         "result": 0
8     }
9 }
10
11 // 创建打印实例,此实例只需创建一次
12 this.nMPrintSocket = new NMPrintSocket(socketData);
13 // 调用流程
14 async DrawLableQrCode(){
15     const DrawLableQrCodeParam = {
16         "x": 20.0,
17         "y": 10.0,
18         "height": 10,
19         "width": 10,
20         "value": "精臣SDK",
21         "rotate": 0,
22         "codeType": 31,
23     }
24
25     const res = await
this.nMPrintSocket.DrawLableQrCode(DrawLableQrCodeParam);
26     if (res.resultAck.result !== 0) {
27         return;
28     }
29     //进行下一步操作,继续绘制或提交
30 }

```

3.4.2 二维码绘制 DrawLableQrCode

代码块

```

1 export default class NMPrintSocket {
2     /**
3     * 绘制带logo的二维码。
4     * @param {*} json - 包含二维码信息的JSON对象。格式如下:
5     * {
6     *     "x": number, // x轴坐标, 单位mm
7     *     "y": number, // y轴坐标, 单位mm
8     *     "height": number, // 二维码高度, 默认宽高一致
9     *     "width": number, // 二维码宽度, 单位mm
10    *     "value": string, // 二维码内容
11    *     "codeType": number, // 条码类型:
12    *                          // 31: QR_CODE
13    *                          // 32: PDF417
14    *                          // 33: DATA_MATRIX

```

```

15      *          // 34: AZTEC
16      *      "rotate": number, // 旋转角度, 仅支持0、90、180、270
17      *      "correctLevel": 2, // 纠错级别, 取值范围1-4, 默认2
18      *      ""logoBase64": ": string, // logo的base64编码(不含数据头, 如
data:image/png;base64,)
19      *      ""logoPosition": ": 0, // logo的位置, 取值范围0-4, 默认0:居中, 3右下.
20      *      "logoScale": 0.25, // logo占据二维码的比例
21      *  }
22      */
23      DrawLableQrCodeWithLogo(json)
24  }

```

3.5 线条绘制 DrawLableLine

代码块

```

1  export default class NMPrintSocket {
2      /**
3       * 绘制线条。
4       *
5       * @param {Object} json - 包含线条信息的JSON对象。格式如下:
6       * {
7       *   "x": number, // x轴坐标, 单位mm
8       *   "y": number, // y轴坐标, 单位mm
9       *   "height": number, // 线高, 单位mm
10      *   "width": number, // 线宽, 单位mm
11      *   "lineType": number, // 线条类型: 1:实线 2:虚线类型, 虚实比例1:1
12      *   "rotate": number, // 旋转角度, 仅支持0、90、180、270
13      *   "dashwidth": number // 线条为虚线宽度, 【实线段长度, 空线段长度】
14      * }
15      *
16      * @return {Promise} 返回一个 Promise, 解析为绘制线条的结果
17      *
18      * @description
19      * 1. 绘制元素前, 必须先初始化画板
20      */
21      DrawLableLine(json)
22  }

```

代码块

```

1  //返回数据示例
2  {
3      "apiName": "DrawLableLine",
4      "resultAck": {
5          "errorCode": 0,

```

```

6         "info": "draw line success!",
7         "result": 0
8     }
9 }
10
11 // 创建打印实例,此实例只需创建一次
12 this.nMPrintSocket = new NMPrintSocket(socketData);
13 // 调用流程
14 async DrawLableLine(){
15     const DrawLableLineParam = {
16         "x": 2.0,
17         "y": 2.0,
18         "height": 2,
19         "width": 50,
20         "rotate": 0,
21         "lineType": 2,
22         "dashwidth": [1,1],
23     }
24
25     const res = await this.nMPrintSocket.DrawLableLine(DrawLableLineParam);
26     if (res.resultAck.result != 0) {
27         return;
28     }
29     //进行下一步操作,继续绘制或提交
30 }

```

3.6 绘制图形 DrawLableGraph

代码块

```

1 export default class NMPrintSocket {
2     /**
3      * 绘制图形。
4      *
5      * @param {Object} json - 包含绘制图形信息的JSON对象。格式如下:
6      * {
7      *     "x": number, // x轴坐标, 单位mm
8      *     "y": number, // y轴坐标, 单位mm
9      *     "height": number, // 图形高度, 单位mm
10     *     "width": number, // 图形宽度, 单位mm
11     *     "rotate": number, // 旋转角度, 仅支持0、90、180、270
12     *     "cornerRadius": number, // 圆角半径, 单位mm, 暂不生效
13     *     "lineWidth": number, // 线宽, 单位mm
14     *     "lineType": number, // 线条类型: 1:实线 2:虚线类型,虚实比例1:1
15     *     "graphType": number, // 图形类型: 1:圆, 2:椭圆, 3:矩形 4:圆角矩形
16     *     "dashwidth": number // 线条为虚线宽度, 【实线段长度, 空线段长度】

```

```

17     * }
18     *
19     * @return {Promise} 返回一个 Promise, 解析为绘制图形的结果
20     *
21     * @description
22     * 1. 绘制元素前, 必须先初始化画板
23     */
24     DrawLableGraph(json)
25 }

```

代码块

```

1  //返回数据示例
2  {
3      "apiName": "DrawLableGraph",
4      "resultAck": {
5          "errorCode": 0,
6          "info": "draw graph success!",
7          "result": 0
8      }
9  }
10
11 // 创建打印实例, 此实例只需创建一次
12 this.nMPrintSocket = new NMPrintSocket(socketData);
13 // 调用流程
14 async DrawLableGraph(){
15     const DrawLableGraphParam = {
16         "x": 2.0,
17         "y": 5.0,
18         "height": 30,
19         "width": 40,
20         "rotate": 0,
21         "graphType": 3,
22         "cornerRadius": 0,
23         "lineWidth": 4,
24         "lineType": 2,
25         "dashwidth": [1,1],
26     }
27
28     const res = await
this.nMPrintSocket.DrawLableGraph(DrawLableGraphParam);
29     if (res.resultAck.result !== 0) {
30         return;
31     }
32     //进行下一步操作, 继续绘制或提交
33 }

```

3.7 绘制图像 DrawLableImage

代码块

```
1  export default class NMPrintSocket {
2    /**
3     * 绘制图片。
4     *
5     * @param {Object} json - 包含绘制图片信息的JSON对象。格式如下：
6     * {
7     *   "x": number, // x轴坐标, 单位mm
8     *   "y": number, // y轴坐标, 单位mm
9     *   "height": number, // 图片高度, 单位mm
10    *   "width": number, // 图片宽度, 单位mm
11    *   "rotate": number, // 旋转角度, 仅支持0、90、180、270
12    *   "imageProcessingType": number, // 图像处理算法, 默认0
13    *   "imageProcessingValue": number, // 算法参数, 默认127
14    *   "imageData": number, // 图片base64数据, 不含数据头
15    *   // 如原始数据
16    *   // 为“data:image/png;base64,iVBORw0KGgoAAAANSU”
17    *   // 传入的数据需要去除头部, 数据为, “iVBORw0KGgoAAAANSU”
18    * }
19    * @return {Promise} 返回一个 Promise, 解析为绘制图片的结果
20    *
21    * @description
22    * 增加接口说明:
23    * 1. 绘制元素前, 必须先初始化画板
24    */
25    DrawLableImage(json)
26  }
```

代码块

```
1  //返回数据示例
2  {
3    "apiName": "DrawLableImage",
4    "resultAck": {
5      "errorCode": 0,
6      "info": "draw image success!",
7      "result": 0
8    }
9  }
10
11 // 创建打印实例,此实例只需创建一次
12 this.nMPrintSocket = new NMPrintSocket(socketData);
```

```
13 // 调用流程
14 async DrawLableImage(){
15     const DrawLableImageParam = {
16         "x": 2.0,
17         "y": 2.0,
18         "height": 10,
19         "width": 50,
20         "rotate": 0,
21         "imageProcessingType": 0,
22         "imageProcessingValue":127,
23
24         "imageData":"iVBORw0KGgoAAAANSUHEUgAAAZAAAACgCAYAAAAisjrVAAAAAXNSR0IArs4c6QAAAA
25 RnQU1BAACxjwv8YQUAAAAgY0hSTQAAeiYAAICEAAD6AAAAgOgAAHUwAADqYAAAOpgAABdwnLpRPAAAA
26 AlwSFlzAAAAOxAAADsQBlSsOGwAAROVJREFUeF7tnQmYFcW1gFtEDe64Am5BIyhxe/jcRTQquBDc17gb
27 FfdEFUYlKu6JccWnKIoYiIIGNVEB81TcggsEFQFN3BWjKIIILaIIG+9V/pmuoqam+3ffevjPDzPm/r76
28 Z27e6urrvvXXqLHVqia+++ipeuPD7qF27JaM45m+7aIkllpCiKIqiKHEcS/n+++NbGhn/i40ssLIid
29 mzZ8dLLdU+at9+qegHP/hBvQBRFEVRFIsVIP/+97+j//znu2jevPnREvPnfx0vvfRS0VJLLZVUUXRFU
30 ZR0vvvvu2jBgm9RONqp8FAURVFyY2XGEgsXfh9jy1pcWbhwoahW8+fPj/71r38Z1eo/Ih27d0kSLbPM
31 Mmq0UxRFqQHffP0vaAkz+MbJ6yDffvttNGrUqGj8+PHYvw8D9HLLLScD0bYx29zSSy8txxnYGdBD907
32 d0zr66K0jLVZaqX6gf+yxx6L7778/mjt3rrQ3b948aRuJt9pqq0Vvv/12NHv2bLHfffnll/L+CiusIH
33 Xd69xzzz3R/vvvL0JEURRFRKRYm7JkChIH5N7/5TTR8+HA5wYfBncEeVl11VXHCwxdffCED0oJkxRVXD
34 GoCBx10UDRo0CARDPb9Sy+9NLR44otF+Ky++urRj3/8Y/kfofHkk0+KUNh5552lXbSMtdZaKxoxYkT0
35 2muV RUOHDo1++MMfSjuKojQ/LcUC4A5zTz31VLTLLrskrXqSMRw2Ik9bpeo0NeXeXylEHpgGq+aEE06
36 IBw8enLyqY/LkyXHHjh3jV199NTmSj+uuuy7eYYcdYqNpJEFqe00NN2Ijo0Jhw4YlRxZx4403xuYDio
37 0QSY4oitISMONMs5ctt9wy6U0df/vb3xq8/4Mf/CBef/31G9XLg9/WUkstFZuJb402/DrNWYrkm2++i
38 evUhRx89dVX0YwZMyIjEBqUv/71r9H7778fTZs2LXrhRfqjz/zzD0igaA1TJkypcE5lA8//LDef+Gz
39 YMGC6NNPPxUzLQVzFXXRiCguaD2Y1zBn4QNRfEXJC+PQ119/nbyqwwyOMm755fXXX09qhGGMYvxqMyB
40 J8jBy5MhG0qyasueee8YfffRRbD685Ap1oIHsuuuu8fTp02MjDJKjCtXp0qR4LVVWiW+++WaRfC6jRo
41 2K11577fiJJ56IjSBjjiqK0tyEfvtNXXzN4qmnngrWa9++fVIjpp977rlgnbXWWiupUUeadqEaiEeHD
42 h2i5ZdfXjQLc259Yda/7bbbRrfeemuD42Ywj1ZeeeXo2WefbXCcctFFF0XrrLN06qJFroPfI6/9dIMN
43 NpAZxKxZsxp0LYqitDwYL/wxoYhiJqTJFUpD8E7ofDcIJ21pA37ZUthgnzzU6jlQ8j6LasktQBjMl1x
44 ySYmqooMWBAPqHQ5097it59dHXcQERXu+gMAMNWfOnPrILfe8UnTs2DHaZJNNon/+85/JEUVRwiq1Mj
45 Nj9i4KxqkQBAS5bLbZZsl/i8g78a2lub3IZ1GK3ALEwk3bgf3555+Pjj32WImm6tOnT4MHhx2Qem6IL
46 iCE0qKyaBtfCxoFdWxEVxZEYvXo0SMYqmJ9iK+iKC0Td2LIGMJYUGnp3Llz0lKxAzLjT8+ePRUjTba
47 SCbM7vWJBnVh4vvZZ58lr0rjT5CreRbuc4BaCieXsgUIAgDpfOedd0Y77LCDmLZ0P/10MTu5MPgjDBA
48 w/fv3l/Bbyr777hvdddddjRzhwMBP+C9qoP/BlII+bLPNNrKG5L333sutuSiK0ry0b98++a88GCM23H
49 DDaI011ki0FMvGG2/cyIHO+JIF/VpllVWSV+VR6b0AWj2HLMoWIKhGCIXjjz9e/rL+AjX01xY000CA6
50 IEHHhDz0qRJkyRaizJx4kSxz4VURKK5kMLdunWTxI5I1jSYCeyxxx4S5YXA2HvvvWXNyJgxY4LCSVGU
51 lkdeK4MPE0xKmq+iFjAm5aHUuFWKSp8FNOVzcCm7xziIbrvtuuiNN96Ihg0bFi277LLJ043Zb7/9RCD
52 g3EbwUDAXnXvuueIk90HKsyiRmQWaThpI6q5du0rdm2++OfR8889FAh988MGymJCQYkVRWi+E2TIG1Q
53 osGcccc0yDwoQ5C0xHdmF1W6BsAdKpUycZ/BnkKyVknrr55Zej8847Lzruu00inXbaqVED+xqNBmnLX
54 7Qg0p7ccMMN4oAfMGCAmMhIjzJ9+nSpryhK6wNtYP31109eFc/HH38c/f73v29QHnnkkUaRU360ExNf
55 ok/bCmULEPwDNLWp1Fu4Vz8J9bMhLpHfqsjjzxSvRIenwaPiw8JDrr/PPPj9Zee23Jf3XmmWdG662
56 3nmhEL730kgiWgQMHSj1mC2+99VZytqIorQnMPdWYfLJIC8X1/av+okHr1G4r5P4EGJRZDU6kE34M69
57 OopDCwY9ZC3UMNRbBgliJnDH4M/4tBVAMzADQfPthf/OIXYg5DCGH2ImLif//3f+UY57MiHhX30EMPF
58 cGSFpKnKMr-iCdF0mK6LILTq/MUXX0zebQhjoFuP8cyFQKA25YM1EjUX77zzTmwG8dgIKuRI5cyYMSM2
59 Qqh+RbkZ+KWUgq13jRaSvGrI66+/LqvajUBKjsTxzJkz4y+++CJ5pShKc8AQ45ciVmmzatxMJn015a9
```

E90lbdb7eeuvFw4cPry+DBg0K1n0LmfzGHTp0qKhfLT4LSt62iqSsleg4rY866qig87tcWLOx/fbb15
uqWN2ZtcITkxXrPUIQn00ct0t4x8zFGHRFUVonRfka0iKYyOzt0tHJoJEFGkhzRUQ1B7UzIiqKomRgF
7wRRWmLXe1d6hgTRLJfYFKqljSfBdGcrgBhPRum9LL9YpKLmb2SfvnPwrYN/jH72h4r4jlUQuZ+IErb
AV8Ruc2IeCsVnq0oeQkNzltuuaX4UoE0SEcccuQj5zTnlTrGX76vrEH7wx/+IMfwSfz3f/+3/0/iXi9
E2nkhsJpg8XDx+4oWQmqlcvsvVehb2+fnH3NeAVcdeD9Ku6Z9XDYXtB6K0Dj755J04d+/eYlv++9//nh
wtDvPDIs2sLt5vv/3Eb6W0fsw406hsvfXWybvFwp5Boev5/gGftPNCZdtttt030yk+aP6JWzwHS7qlI8
IE0iQbCGo0333wzU81ac801xcfyzzjvJEdKs+6664oqmxdmLIMHD4700ussido69dRTk3fyY2fooa1y
89wns4cf/ehHjXw+zFpYC4NviHbcVa9uVefe42gQLOIMJXvLggWgRLSRuqFITyR7JEszedPIIMD0kkr
rxs6gXZjB29ky71cTjsvwxfcKmMGzHMDHn+kTdUVdC9k080/mwW+L35qZbCWvFvWH1E7du3eXY2nagP
scoJpn4T4HSHsWRQ73TaaBvPfeew2kYFphj5Azzzwz+F6onHzyyckV8kPEhfmgYv0hJkfyQyTYYYcdJ
tf+7W9/mxxdxMcffxb1bVRP/1y2223JWcswnz4sgfK5ptvHt97773J0UUYQdBorxMi4jju8orr8Rb
bLFFvRyXZ84cea5GONWxN/zkJ/Hee+8d77TTTvXH2AmSnR3NF1t2f+zZs2f9e2awj/v37y/nufXP0ee
csqLd5s6dK7u2FRHNp7R8Qt//pi6+BhKKumL3VdfqKi3iKk9bFHffkDQNpDLkkZQVhVUNLPYz1ytZyJ
1lBidZVX7hhRdGhxySLAexc4A3H2Gkb5//OMfRYqXKmbgkzaYSYTed8tDDz2UtF4HkWD33HNPnHbsW
FmwyJ7sU6d0Td6NZlB0PicXXHBBBoz5TZs6cKe2GosN4H80D7Yt7Rx0zheRszEy4nts/IuM4zvu2Ln0w
wkPyitGebZuYeVbX2sI+LdwHKRvsMVbVch60QjQ7kmHa9/7xj3/I8yAHmT1GIfo+Mfl8fm7f0gp9pT4
ztND7frnqqqvKhhSLKPwoKV7zmySS0zrM86QtAb8tfo9kyciKKm0tNIkJi0uwVwcL/A488MDk6CIYhP
/rv/5LUgUYLSS65pprovuu08EAqF0PldffXvKNIBo3LhxkoUXuAYLDhkY06D0008/LfmzzjjjjKhXr
17J041h8CKlCpEeaZA52A0l/NOf/iTmH/r3P//zP8nRRWASwnzD39122y052pBPPvLEThlpG01ChB8R
G/QxDUxQRGikwUKpfv36iXqNKTCLxx9/PNp9991L4RRZmH1QmV2zgAv9RGAghMhkYBkxYoErIggHID
zUP0RysriSanvaFOByRjTssU3KWH+RXgwubGTU34fpC3x8dsK1WM5AZM7Wy+treagy0Ge322TCRCEBx
lzGbQRDDa3Fe8NGTIkMmqjZ09l5ssgj3bBjPenP/2p1APqMuPFXsmHzup0tIJy+POf/yyzZdrea6+9k
q0l8X0b/CgQbHzpWKHPandmyo8++mh0zjnnRL/73e+kngt9Z3ZP330hh1AgZPDkk08WgYRdFTttCGbv
F04/zV7Ke0SL4K9BUyJ5pQ8fVn20D0Slfug8cz47PpuRI0cGU81kgTZDG9ttt1107bXXRr/85S+jDz7
4QCYKSuuFyU5zQ2r2rAglfktMYq0AYTJERJSP31aoHr5WNworra3mgHsviiAPwnr33Xffto4d3bJgwY
L4Zz/7Wfzzn/9cbPz4AqhnPuD4oosuSmotwkhzseVdeumlUrdcnn/++djMEMrygRjtSa7pFqNjIR0fm
74Zi0Pu3bvHZnaenFEaBTjx4+PjcCMjWBtsII+C+pzfd8fUglGE4pXXHFF+VsKbMJc02hYyZHyMJph
bASu7Hdvzn9/zz77bPG5KEpT4vsklllmgff/+P4NpTT4QJpUgICZYuDl1BRnOukNGHPw0qhQH1kks
uEUfUW2+9lRyNZdBE2PBhc66F/0lt4H4piirLLbechLe6IDxw4PMALQhCQucmTZok9xQqONxefvllqW
e0i0TMhpjZSzxgwAC5ttHS4mWXXbZB4ZkQsrj66qs3eo8fAufhBPf7nIbRlmKjukt4bYhHHnlE2mTwr
wTu2WhVsdEy5fUzzzwjnxWCEHi0ZuYnjin5FaQp8AWK08Lhr164qQMqE8a9ZFhLiD8F3gQ8BZxPhbqNH
j673J9ClV199VcWdZN/FqQ7/93//F/Xt21dSCHgBI8eAjJiontj9cSRjlsHchKnHhXazfCA4kDGLce7
XX38tfoSttttqq3uSgn4XwOFJJ46vBnIMphteornkh5Tw7M/oQDHDZZZeJMx0TXblwrhm0xdSEfyEL7p
Okkw8//LD4KLdDwqzPBr/Ovffew3aKBkyV7NFy/fXxSwJM4Pnj/+EvAQ1cHzMlzvkZM2Y0uL6y+INDO
guCbNzfm4EdgwYNSl6F4XfC755QWhtu2nl++3z3QiHk/PbtmJGnDxa/fZ+8/SqStGuGxpxKaXITlg/m
C90P2AiG5EhDzCAo7xPWiumKWtAhvsz205g3b54kfkWDMwmlLFjxyZHGvPxxx/HRngkrxrCzH6PPfa
ITznllAYaCJx33nnxAQcckLwKc/rpp8dG2KQu1EMDMR+8mJYI0SWUNm8hBBjNbq+99hKzUTkMHDhQnv
XgwYPlHjEp8hoNpVyM8JawX86fPHlycrQuVPmpp54SDQSNzOWCCy6Q+jxDpfXAZ5pV1lllnaR2HWmhs
X5Bs3XPTTvPbz8PeftAyWq/yH7lJe2aRcL412y5sJjt2lTIF198cX0eGBccTziBDzvsMJm1MjtlAaDV
Bnxw1HI02sCECROSo4tgmId4aHffrzPYj5SwqMhoIGUwndiE3mFs33TTTcNRhbts88+ov0QCmwXGZU
Cx3YlpRIuv/xy0VzQYIgC47NhFkNQDkQSEBIJCn7CR12nZX0zYYw+6HmV1xxRXTLLbDEv/nNb+TzQx
tR2ga+1uk7uENGZWAG756bdL4lWm2ePliy2k8bryrpV17K6X9VJMKkSRk3bpxIQ2z9RpCIJ07WrVtw1
vzQQw/Jwj/qhxbvubDQj5mzGdiDDm1m96SkRwMxg3hydBHMk01iPmzzIS3EaiA4gIH+o43MmjVLXgP3
tcIKK8S33nqrpJk3g6L033yomZoBfTRqbXzkkUcmR8oD7aVcDWT27NnxDTfcIAE09JPPg78bbLBBPhr
06FR/jQWNEEc8mhXn0Ra8//77MhOyPiA+EzMBkM+HZ20PU/AdEZTA8zICuL4faIrLBBkoLQs+x5ZYOn
XqLPswPG0jb3Hb53sfqhMq7nkhqu1rkTSLEx2TCDdy4okn1g9Mr776arz88suL2cMew8RjNAGpe/DBB
0vUFF/jWB41apTU8cF8dcIJJ4iJBEEQ4v777xcB8vDDDydHGkLEFc60kIkKuAa5nIkg690nj/TptNNO

S95dxN133y3vUejPk08+mbxTGgSI0chktTb0bQb1vIX60ATzCBAG5Ztuuinu0aNHfT+JeuP+ged/zDH
H1L/HivXQCnkW2pZ8Pttss40IVGA/Fntu3oIznfsHhBCfP+ZNFSCLL6HPudKy7rrrxnfcCyEUCy+8ML
N0qXpuBgcmL6E6edvyizU/W/gt3nnnnbnaCmWwCEnra6j4/acUSZMJEC7ETHKNNdaQG0MDCcFgwcDLQ
Eg9BiQ7oFl++ctf1j+gfffdN3700UclsgusAANcCnPPvts/Otf/1reZ6buv89M4Q9/+E084YYbBgUI
g9uDDz5YH/HF9S1EiyHYrFChoHEcfvjhMqvgmvhvmNETplyKm2++uWINBOFz0EEHJa/qoN9EPxFJha/
E9o/oN+7Tf8Y+PBe0LnsepW/fvpKShXDcckKK2RQM4Zi1gZjS0nC/M9UWN5HheynpkUgG6oJVIVRv++
23T2rUWRVCddzrQdo13cJ3Gz9kVkrXWr+yzkvra6j4z6JomkyAYFo6/vjjZeb0A/DBbHHEEUfU3zg5l
myYZwic3EhwZtvMV01AT04mHpr7ECstr03wTVgMxI8//ni84447xk8//bQ4hN0ZPIUQWvpEqK4LYcr7
779/g7qd03e0zzrrLAnt5Z7Jx4Nj0L6PGSykaaSVlVZaqf5cwo8RxDyPqV0nxtdcc40cRzthXcebb76
Z9Kw8XnrppfjKK68UkyPtYQq0AjwPPD/OQ6AprR/7fSyiMHGz8NsL1eE7f/TRR9eXfv36ZdYjJ1yojn
s98MN/QwVz05NFVxAwYXT7REnrV6XnhYr/LChF0mQCJA9sczts2LDkVfNy0kkniakN00wW1157bXzLL
beUvX0uiwhJR0j6Tlo7zLoQQAhgzJaKwuQxNPj5s/885G0rVI+FrlnXzCNAEDoM3K4gKMdvkee8PM8m
7VkUCQKk2aKwfNgQ5bjjjkteNS+33nqrRB8R6ZEFqeFJQeJHFwXB2gpSnpTKe9XaIAqLdBGkciFSTVF
YuxXCjE/Jf/lJa8uP8PTrsTUD1wtFgpYL33GKS1oUVhZp5+V5NqyRaQpajABRFKX1Qe4lv7hJN9MGbc
LsQ+fawsJh9rtx2yIvXQhy0Ln411y4cKGcmzUwk6MuC5YC+PXynFcOWc+G8tdkiUst0S1tFUWpGf5sH
Ng3/MMPP5T/GeyqWbPgTKXSTzIq+Ky66qoNkrL611xyySUlg8WGG25Yn0wxRKH9hJ074rtdu3ai0WBR
sw3l0c+ypbNhVbXPhnUy/jXyZAbIC2vdVIAorQK+xqSh+0abb2QwKDftilIbQgKE/WpYsAuVDpJ2ISG
phGxbefGvyYDfQVMnSf1TSOCEII0R/bDwvVttxRUlW3eptvzzLEUKkG233Va0tFqBAFETltIqCzBvg
tsSkbhB1o0ZCZgJnnKKaFIKn2lMlxTU6V+B7SG9u3by99qQcjRFhOPcmFLB5+Q0PRhM7YQbh82q2A7a
pdKfS/loBpIBjwe9tUguSF/0+DDsruQkSwyC2Y9zEDKcaLTlQldSLLIPhoHHXRQ8k4+sPUyQw/toZL3
PpnxbBDBBrLrmguDAn06/vjj5TU/SMaMTLHY4/TF/erZ43ZAYSy2atQoMSuUwyeffBKdcMIJ8lkMGzY
s0VoMJP+84447ohtvvFH2lenZs2fyjpIGn60Pu6dGaK8MvqPuPuNPIDzQZuyMndk2CTp90C7cAbvamX
0WCBC+G6U0kLz3TVtoWyzNPI+LyhyuJd0TaZBpQSScRgzZgxPPbOwsHD33XcPvhcqv199dXKV/LB4j
3T4rIUpBzMwy3oTQmhZi0KiShfSkbBGgxDEUF9tIRMAiyL9zJdYQp0JqWfdiQvx7KGFkSTIJPWMD+0c
csghsnCLzAT0m/MJcaSYH3/cq1cvSca51VZbNTj0ay0YYy0YJSWKPC5977bbbvF2221XX5/zCdcuZyH
kX/7ylwaL0JTiyZPFgEWwrLnic7SkrdL2V3ebgT1Yr4jCQkK2Wth6662Tq+WH9XJ+eySQzQrbDZ2XVo
qE37yasDJAUA2AHQ/08Usvs2bNlD2WSPpIo8Ve/+pXsUBiQsyGLmltjurOgzz77TLZ7ZcZRqpx00kmyg
yGqc+h9t5Ai3WJna6Symp9m4JZdH+22m7xPKDLmH2aHoX6z5TCJDkMaDFoGmgsp4DfffHPRYcjMnkjd
z6wP9dztnxG40cSJE2W7W1sfJ+T+++8v2hbaCndFW2G2Q7JMCmGYzz33XPTkk09Gc+f0rT90dAp9QDv
i+RjBIMfRHEjOyLa806dPr69PXcId2VGS5JZu39IK2wlw7dB7oVJk+uy2Ar8nnh2/PVv4brjHzMDaaG
dMvjuh8/L6w9xz/euV0ubC95XvFN8vvv02uNFignbA79gtvq0da/Hdx7dnIaIrdF6evlKKRk1Y0eALQ
XZZvtj+PgIMbOyVwZo0TCbrrL00mDiIDGG/kC5duiQ160CVvvLKK+WDN9qERGSag+VHH30ka1waDJBs
xcs6CtaesKYiDRx5D0Shwd7CA0v+AC699NLOhhtukIzHhx9+eHK0Dq59++23R4MHDxYhkZZNmGdV6os
6YsQIaafU3vXAj6bUD592K0xDjwDKgvsaoNso/NgwwbnQ5zfeeCMY9/kyCL4ESJc+Wu0D9m0FzCtsS
8/z5nzQ9AvPt88+7IoiojbryMEpk87wObZ5wNqbcIK4UaLQZq5LYTRsjLNdHkpcrhXE1Z0zGxCEpGtv
fbakosLs4qFFZ+kVSEVC2lazEARsQrJp+Wn68BUQs4tcmKde+65cdqeI2mY2YikESHJI0p6Hlj9TZ8w
A9EfCueSgRcwhZmBuD4lCznEQmDKw8RlM+7a0rVrV1kxSw4uTG0o76RgQZX3i/khS5oHzgu9TZHCR1K
ykJqFZ0j2ZB/uiZXsJNnkuqXAdMc9kyqGPUe4j0rgHskHhimNLMHs60J7yuKHb8Li05dnJXoIcru5ba
UVzMa4u7JMTqucfWfuiYsFwzXdp5lfs9WtSELrNm8TiHwAh0LntGiCTvRGJCeffdd2UT/VVWwUVM3926d
ZMwPvY8cWEWHzBYWaKGL40q0jMZtEa0HTyWmWZUw/ayk477SRl6623Fu0Ixzdz4MPPigmtbffflltM
ZCG4r1133VXuadq0adETTzwhJqHx48fLzPhEm9Me5iXUd7Qbv3D/7PnBKvTQ+xRUdEx0mPTQCo466qi
kB4vgnni+vXv3jsa0HSszyjTQDjC9cZ9GcFekxnOff0aHHHKIPD/WFdAWn/2pp56a1FIWF2yghoXvCF
pKJd8N38KQhht5BnyH80A9NzILq0KISvpeCIkwUXJCLqeNN95YnLgkTBw6dKjMLnAIA7Nj9sFgPxGct
jgELa+99po4mSk2BxYzYlLM81HUojAzMaqmXAvMoBcfdthhsreK0xtHy2JfddQkq6n4hfe4/1I7PpKk
kfvm2ubL30C7wPGJxkLadrQM9z20E9vnffBZR/qZhRFW8S9+8QtXkIcyHHPfaGxkQyYQoLz4LPL8zeS
gPoM0ec/YL4bPkizGaCSHHnqoZIJWGM/0+Ys/h4bfHZ2F09K9+7dg+fLLXyv3fZC5Wc/+1ly9Tqqce
T71y0YIFTPv++iQQNRAVIBpJ0nygf1EjPLGWec0WAAwTRfLBMbMt1111yDLMXgzamGTdpJGap0XPmy

0BMhA+p0zGX8D8p8N1C5lsGq7XWwIu+/PLL73PIIEzibbYf4RNnkhSaU1u/KV9BmgIwBAglQow9v8I
QeQU29Lee00NyZHywLTH88TslofXX39doqlQ8xEoLnYjL0xNP0dyIeU/eyoQFQb0iVT7CBCbDBKhS2Q
XkV7lb0LVVvC/N81RsvbYqGTvGrdUYj6qRoD410vrF9Z9V4sKkCpgps2g9aMf/Uhm5T5k58U3woybsF
d8KAGddjLELp8GvgnsqiHy+kDwaxi1PHm1iG+//baRALEgAEktT/uLYMbdv3//BpqVC9oXe7KwJwIzd
77EeQszQUJwzzzzzNwCBKiLDwqtBsHNs9l1111FwNvBvxzYSoC+4xdCIFpCAGTs9ZkZ8r6yiNDA1tTF
9SGE4Pcb0i9vyWo/RDn7evgFf54LY0aoXq3DzdUHUgWTJ08W2zvRWexzbp5n8k4dhMQ0GDBAokIIS2W
/dGz2ZoYdXD1LuB7t7bHHHrInu426cMEvQPgdfET+LBS2gygksLgoosuqouSKMFSTpQTEUP0kzDkgw
8+OCIs14UwRNo1A6f4T9Zee+3kndJgw81TqoFIK077pptukLXiHA2vvPLKEjVH1uNyYE927pNILXxVb
roJrsNz4v0jfQvHCdcLMg6fSJ8+fcTnoiwe+N/1cpk6dWryXxiyFpCDyi12wW0l8Ft022IJQQgWF7r1
KIVTJ0uUvDCLxzTC5k/M1tmznVkzs3p/1owWgNmKx0xJ24oX0C6Y1f70pz+V2bi/GA+YtbCjIjZPzGg
+zAiYceMT8H0cYDUQNvbClg9PPPGEmM0szwHfDDsacg36S/TWcccdJ/03X8BGbfqggWDCYL0tSsCEVa
4Ggk2bc9AA8Keg6dFfZmqYAkVbc8f0heZgBIFoLY888oi8x7a+rg+I3S+Jwr0mSXucnS75vPAjSDGrV
T4zQN4rpXG2dux33y2VmHzyQERk6HpZGki55qRK+o5Juqi20u4xbykSNWGVcb4NI+0lvI4VzBa2qWW7
XjaiSjCQI2T40DDps0Ka//GZsJuhDwMZ9nTMQzjGPvjgg+SdReDDsCYs67T3QSCwC6FvogIGMgQCQor
95zHRcC+hwZ7wWPuLw2TDlr15QICw+6SZpctgXE5hh0LMX3kECM8HYWNXzuNbYpX+/Pnz5X0GdjcrAF
kCeDYh3n77bREM7Nhm/Sh8fvbcvIVdLAGTAuZLPkvCLLWANCz4i2rBK6+8ErXeLgAhGCZ0XqhU0vc0A
VVJW2n3mLcUCQJEFxJmgHpLaCqmHT0oiumGhYNGS0hq1IH5iBz8JPSjHrBBFovyzMxYXm0CwsRCKDCg
xLKHru2EEZqZtOS6YtW0GYBk1bfl559/Ht1//2RmeVGZpCnTtpqq+Sd0ghHnDRpkoTLyYoY644wzGoT
3kVaaxXQXXHCbmI2Mni0LB4GFjYT7ssjPrprF/IZ5hvuibTMgRoceeQj0t1T4IgvSWIF0H8vl/PPPl+
eFCc6CuY4+GKEgecd4PIBQXkxG3Ke/ItiF58FCTzcz6YEHhj169dPwpv9z7IUY8e0ja644gpZja5kE
zJPMgG93kSLSDzfPZ0Hhi3CwjEL2jxRaQsE3euBf03+P/LII5NXpSHHmv3+5SVvv/IwceLEaMcdd0xe
lU+Rwz0mchUgGSBAGNCvu+46SXzIwGZmy8m7dau5+YIwGE+YMEG0sXaBev6KZwuCgtXqrJxlACelB3Z
1vkysACehYbWwLoOUKgzKfGZikgDST9Z7YCNFYM2aNSupEUkqE5IRmtm7rHcAviicR59fe0EF0Qasd+
GHT9FGG8n16Lfrd0F4hfw9afDDtuCfwY+ET+jEE0+UxIoIN65Hahn8FLZ/eWhAwXf6nWjNYrQJhsAg
tL1B6XB+hY3mSKpYZTSZAmQaldW59lxbX+o814T/5e/nwbFOXdvkTwUKUDS9jzJS5F+EPmtI0CU6sDX
Yb4kEkLb3ODDwBxz9913By0xXKhrNKTYDKjJkXxgGi0kmH0x+7cVMI9h2sIXlNek19Yx40yYjYgb05N1
8K7JDBZMnkXLUcm8zGAfruteDvNfEFFwEaRFX+MgWZ9SEpShKTcnSQNJm51mQLQFTJ9pvVlv+TD/vNc
ks4WsbacWXXHJJ8qouwsrXUnzQGsG4J0nfZ+062WdVwtUA1GUFgAr3lmHQxAfDv3WhBlNghVXI0jTG
vIUAjXyt0VrINVckzxoLmkRVpUWv32ft0tlnVcL0EB0HYiieDCzssWuESB/kj3273//W44BPrDQ8XJg
ls40igQD4PNx/UxFgX+JTM5777137s2HWjLkhyKfXFPj70BYdKbl0A6HLmnXyzqvVqgJqxXD4EfKf4k
PQ7skMnBRWByHKQBHMLfCwIDpfzWoI10c5JikZCQpYL7Ye4PAAtKq48AmMWE5EAX2/vvvi+Pbj07jPj
ETEJSAQ5WFm7f6TcmD/YJwWF09JYLgz8mBvpEgIRNWEC7tMezI3273XESZzwBA1yTRZ84+4E+sr8Kw
RYM1jzTcsBJz2JErkviTnexYrXQ57vvvlsWtPJ9KCfyrBp49j5FmLCIus0hTTRgVlu+CauaSCY+U7ub
IpQTWZUHv32ft0tlnWfhwRQFvxSVIK0YBg0EAUGqoZxrLoT9IkJuvPP05EhpCGULHDgvfM0QHhRCcRn
I84KAwpZLVBkhuQyCZMdlNoZgYMBLuyi2+EXILIIQb07VBc0B4cM9MYCzMZiFcN3zzjtPvPrjZ7bYqD
T+unuuEKVFHwnfXnfdSWCj/5awUx/mT0THcdnwmEVjgTDs4PkzZ5TX2EGq+pz3vAZ0U02mWXXdbg+
qV44IEH5F4efFTRRkK4VtRKgDCRQMDymZQRQCqNZOK7leXvSIN+uudW01Y1FDncqwBRZMAibHefffaR
QioPBLT+hhgzZowM3ggBQn7hy+/LDQirF0pCtakhIQMg99ZZ50lAyCbW/GXrzDpJAhJZn1HKLUD60g
IkWbTL0KUXRjEWVtDuDCaFwM/MHgZOGMeQLMg/JP32UyLEGY0ELl5o3ka7ZA+hfdt4c0HS8p9BB9pTu
z0j/zoSIdP6DTPD0FgYbC163Zok/vieqTZZ3Ds2bNnvVZDPdLo9+rVS3aXZFZdNaJDaLJuQK0ECEKYz
wnhWq4AqRTMj04kohwI03fXIVXTVjUULUBoUGnFEHLLqnVWafuWsp50JWymY77ckk32oosukhXcofTo
rNJmLfg222wj6UMsrKinFQrpPNIKKVTiskuaDzbcCtWxxajp4qDLixm4JUmlGexjo2EkRxeBg5r7J1V
JKGSZcGdCLbknnlcahC0bwZGaSBII9aUdVtLTrxBm9it7ZoeyDYQgUwBJ00ng6WMEjCTfDD1HyvTp08
U5b2a7jBxSSMVC5mZWYJM+J3SevYdqsdd0ixnQk3frEleG6uQpbPDGc7EYIRGs516vGkgFFGo/T3H7C
Wl9rXUpEg3jbQOGTTC7xvzDzHjIkCFiowf8I8zimR3z1lkdI+wwTVGPTZMsZiQxE7FSnJnw5Zdf3mCV
ex5IOsmKcFb15108xqpfbPYkJ2Q2i0bACnz8FPgn0IgwFzGjZ8bsmp8s9J3Fm2gutGXBf4K5yW4vTFg
o5jDq+3DMCBkpaAHuYLIL/c00YjUJNBosVLrwc6PfZCugzvXXX99gsacL18Shzor+vfbas86h/XJhK1

5Mh/hr8IOgsfBc2Z7X1YBqQZYGgk3/iC00kP/LABMmnxW+HGv7z6uBVLt4MQtMj/ho7Ap54H/XR5HWV
76D7nk+BENUeWRR5HCvGkgbgpTV5LQiDxaaBzADNQ0c5NdCi2DGjLaCBuLm+gLzZZF9TMg5ZbeZNT9i
+ZsH6l533XWYvwZ7leQFzYU07eb7Wl/Qgpjps7EVKejZXtbdj6UU1CNUlPk5ienKgeeElLnQ6088Cz
Z4IqFcDZxYwjukXtLswBS1FcCGpcZqGTPF3K2MRPm+ZOU84ADDqj/LtQK930zpSiNwCevBsKzDNUrop
DMk3xuWYkS0/qadV6le5cUiWogbQxm8cz2sL8yW2dWzhaz+DOYpQEORtK2kPOJaClm23xF2MqWMFNM5
DjaiYDB5o9vwfpCioYZGtqT3dKTGTlpYugTGpXVgEHpgrOaSC/CakNfaWaFaA30G3t0KB090Vbk1sIx
zvNwNSxm/mhua6yxhmy1i+/IBQc3z5Ntf3luWeGdzN7QnEaMGCHanh8VxL2QU4zPieeNBliO+FDsZ4t
/C18J1yJ1Dl0mvi0cuDw7Ir7w69SCKAbC7L8WPhu0Kqtdu/gaSKV+lzzYFDw8T34baaT1YYcddih5HL
YDGw1YCUU093yHVVY0Mdjj4uijj5Z9PfixkRwRc5Q1ozaAE5F02mmnST3MWZgKGMwomKGSyYGVdiYyB
BMDfj8cVHe+5AgXFwbE++67T6KgyBvG3uIu1uxD8kbaZICmPRzYDP70gt5jxuGHhwDBYU3ySqLMc0Tn
hYHTJrx0YQDCpMT1+VsOmBbY64Xnd/XVV+cKwcUcRoTWs88+K8+F0GjgXglIIJDhquuknstB605yPm
EJvN5sT8Jz5NEnAgQzH7k8cLsx6SAwZXR+8k5iyAkQEIRsIsghbcV3UwoQvpNMNjANuveIw9xdKc5vLB
QkkvVs0u4xL0ULEBpU2hjkwCIPD+aokDkJpZJ05y5dusSDBw+OzWAamwExHjhwYFIjDA5RHMRGU0iON
ARnMCys9swIgxMjL0tpzmLmcJjgzA+xkbP7iuii3QS16F0eSQ2RLYIIBQuBIx9HPfZMHib3d8xae
J85pUvd//vnnSYv5wMHN82WbYJ6fEa6ybfgECROSGvnBJIaZiv6494k5DNMjfSSlvwup543gf10gGaC
So8Vgxp1mL74JyzcfmYmLBJJkmY9C5N2fI89K8bxtVdLPWoAJS1eit0GYwbC+gdnMLVdeKX9d0CJYrE
SqeWaoHMfiDMaBHoIZM469c845R3YAdGd7FruGwXzvZIbsw6wfbZfZQjHt8DoNq624EL6LQ5bQVj+bM
WngcbrPnDlTtINSJgD6Zx20mMzyFupzXiWgDWHGYk0Hs0/uD8f+LrvsktTIhmeLNoM2hnaHici9T8yA
zG7R5nwzD1oJz5w+sJvj448/nrzT0pg7d27yXx2+doz2jPZJCHa5hBbohsgT/JC3rUr6WSvUhNWGMz
DAIrNG8HBYIuQA0z2ZuYq/wORQkRmETUFDDKs/whF7TBwsh6CRWpmdltvInFBgHBd7PIMVKTUc0FHZE
DPYEo6D9fHAdaEhRKAHwYRTvhpuB4mMQZH+sH5XJ81HwiNkSNHRuPGjRPFbIa4kEnFwiBctBJmCMx65
YIAZTV5XhMW98xgTXQ098LAztoGTCz4UEi5z+JIf+W8hZ8ugwl+Dp4Lz4AIN/wpRJchS03PG9Ma608e
e+wx8SPZ/Vx4HpgNEToIFyK0MK/wXeBZ903bt6LIL0s5C0ZrBf4Iouws+PN4thYmTHxeZCpw6+XBbys
Nvw8himyrKVATVhuCtQJE3Sy77LLx0KFDk6N1UU5EZ5mBQ16b2bSs0+jQoUNsBhcxq1xzzTXymnUUmL
R8z0Au6c2NFidRjNKim9hGF1MBpq4QmKh69eoVNFFxjRkzZsR9+vSJN910UzHTsAukERBJjUWYmXi9u
o85Km8EE+tgMOPwjDAhYXbIWzp37iz9MYItM8U9pikj4GIjJkWPPH/SxNt1IzxLoq/sPbDF8bXXXism
Nh92P8TEyD3b6xrto1Hkwlah38Bah+HDh8trnjfPXVFCaBRWK4dZ045aoniYfaMVSCqa9SAuaCbjx4+
X1d/sWaiYrTCpuDN2tBAc0HxldtttN9ldEW2BGRyzXWbgbFbF6mU/yonZCs5iIkzIOcWaBBfMQGgxZH
7RHnwNBA2GVerCA9MMazrOPvtsMb9h+sJkg6ZBigjWitAGs3hm+ERHsV6AfeiyALg2GxGhabhgNkMzQ
zPgeZULGhrOdDQt+9zQEB5++GGZ+bMeg10lgb5grq0fpbBrVEhDgoYE0NuJ9uIz6N27t2g0eeAZobmh
yeU9R1HSUA2klcPs0Qyo4vxmXYed4Vp4zR7LRgjUz0TZZzw007XgmGU9hxnA4rFjx8qMlesY9Tvu27d
vfTvVFNZ2mC9ncsU6m02YATg2AkRWSbPinCAA9zz2JmCJT10XZuRoWG7dNddcUzQd2txxxx1FgzCCSb
Qu3keb4Fje4vaFc1m7QX+mTJkiGhyaGc5q1tn4/csL60/uuOMOWaeDk5z78j/TNNB00ADRRuiDoLSLa
iCKaBP40wi9ZX/w5gYfCTZ/ZuiltplFW0C7YF0HvpxyYCUyocc4rNGe2gJoVWgz5BILaYiKui5oICpA
FEVRLlJBgGgYr6IoilIRKKAURVGUiLABoiKoLSEChCLTYIzmVXzhNyGXluyjtvit5NW8l7PL2nnpfX
DJ+16Pmnt++dltZfVTtHX8Uvedv2Sdf3m0i+tnl/Szktrp2pwotcCmm4Jxd8UJ1SHUusnm0aF2KG5bof
eboygNYWEez+Wwww4LvrZkHbfFbyet5L2eX9LOS+uHT9r1fNLa98/Lai+rnaKv45e87fol6/rNdV5aP
b+knZfWTjUQxtvqNRB/wZRNDW7p0aOHLcIrZGGV3xaL01ha5rbl11FaBnZDKDL+gv/aknXc4reTRt7r
+aSdl9YPn7Tr+aS175+X1V5W00Vfxydvuz5Z12+u89Lq+aSdl9Z0tbQ5ExY5gVxIYEY0JSNqky0RrNZ
mHwa/+EnYfFjR7bajKiRsmmlzAsQXAgz6Pscee2x0wgknNCqlMsRCKMusoihKa6XJBQgz9FoU8h/lIX
Quxc0ym2Z28lOI+1gTVh5CfSiq5H0WiQIo1dDkAsTfCrQoKt2LIURaH7PSXmSZuFqx9RygyGehKIqSR
r0asJitv1PYe6EWkGWWfEFuYU8E//q+ECAzLBPAJfhtl1Nq9RwURVFK0awCpNIIJfZurjRyKg/sI+07
002e4aVYY401ZECvhGqitTQ1t6IozcFi6URn74iWiJq0FEVpSyyWAoQNcWo5WG+//faS2twteXwWbPh
TqQLLURRLcW0xFCA9e/ZstJtckbBfBHsnuMWNcrLFj3bq1KLtXSYsRVGUxY3FUoCwVWstcUN6S4EQcf
FfK4qitGYWSwHCDnpFDdahVef+anWL71z3682bN0+FiKIobYbFUoDkiYjKS2jVOYLhhRdeiKZ0nVpf0
nToEJ122mmN6rLoNJSiKG2JxVKAdOzYMfmvetLCZ7fZZptos802qy95hENb2V9bURQFmLWAENm04oor
RiuvvHJ9gaxjOK8xFdUyEmvzzTdvFIVFJstS/Vp1lVXkWcx98p+FxT8WejYaPqwoSnPQrAKEFd/9+/e

P+vXrV19Y9Z11bM0115RjtZzxEysR2HtueeeJfuFoKm0X/6zoB2/ff+1Paaaj6IozUGTCxDXZIRTeu
TIkQ0KzumsY2PGjJFjbnLDpgi fHTVqVIN++P0aPXp0o36l4Zv0/GdB0377/mt7zL+ehhIritIUNLkAC
aUJKaKkRU75+FFX//rXv5J3svEXE/pt3XbbbfI3T1LFWj0HSt5noSiKUg1NLkD8SKaiSt5B04+6sov/
Jk6cGL300ktSiLjKg9/WgAED5G/WviFQq+dAUQGikEpT0Kw+kObANx3xGgGyvZ0+J08qd78tK4xquUp
eURSlpdDqR7qsCCVWteMgd81TvqnK4rfl15s1a1aj6LC0thRFURZ3aiZaBBRRc5esCKWZM2c2Wj0e2g
8k1JZf76ijjimpUL62tpi6Koi iFYwbPNsVyyy2HtKgv3bt3j9u1axfPnz8/qaG0BYYMGSkf/4ABA4KvL
VnHbfHbSSt5r+eXtPPS+uGTdj2ftPb987Lay2qn60v4JW+7fsm6fnOd1lBPL2nnpbVTDd98803c5oz1
voMZhzcMJ/M8ki0Koi hKHmomQKxD0qsYKZacUYfrzE4rm2yyifx1z007z2/fD7F9//33k/8WkacPtvj
t++TtV5GErqkoilI0NRMg7iruUsXPMRXai8Mv7EjIX/fctPP89tE0QsVN4Z6nD7Zk5cjK268iCV1TUR
SlaJrdhOWvovZDY0Mw2BMq656bd16eVeE+efpgyWo/LXNwJf3KSzn9VxRFqZRmFyCsm3BLWtgrAs0WK
V0miN/CP8+tQwG3ji1ffWvAeh99PaChE6320fHwvH/Lb8cyjueSFC54RKOf1XFEWpLMVmZJkwYYKY
Zihps/qnn346mjRpUn1JW1G+1FJLJf+lz9bd65W6pkvnpz1LAHFNU90mTysmT56cq19ZZq1yNAv/WSi
KohTNYiNAevfuHW299dZS0hYH7rjjjtGWw25ZX9IG5DypStzrUfL4LEL9wuHv9qLUv7JA00XFfxaKoi
hFs9gIkPnz5yf/NQ7FtYT28Ajh1kur414vL59++qk45F1CEVFp13Sp9DyL/ywURVGKpmYCxi9yChU2h
goRqutGSYXep63QHh5Z9TbaaCPxEfj130vlxebCcsH85faJQvtZpJ2Xp4SehaIoStE0qwaSzoqqhLQU
6r4m4V+TXFiQNbU39UrB401T6f7t1URShfqhKIpsNDUTIKF9KihF7JUxZMiQ6Pbbb2/QVloK9azB9LP
PPsslyPKkaJ83b16j6+U5rxzy7C0i6dwVRWkKaiZAQvtUUPIMqfL7ZZCg8MQTT2zQFmYau5+HW7Jm8k
Rk5XF0h9r3nfEh53ie88ohzz4iCBB3fx0Koi hK0TS5CauINQk9e/ZstL7BdxrbknW9lVZaSQSIG9obI
tS+LzCWXXbZRsIodF5an7L6UA6+E15RFKVomlyAu0aicqKKXPKYwflCgi/ZKSvtewjfxJXXp5N2324f
1I+hKEpLp2YCJLQnBSVrr4z27dsn76bzxhtvNBqsEQShkrW6m7YYrEPnZhVfEMyZmyfXwJ92327bdq/
2o48+uLFdt6Q9L7ctiqIoStHUTICenLSUNwdUYCGcJ2qJxXm+GSJN11Hpor1KsGG8WX6X0H37fhEbRn
zXXXc1quuWavwpiqIo1dDkJqwsFixYILNq/AG2gHuM/9FA3Nk+juPQeXlNQf55WX2wx1zQdrieLyB8k
1tIKHDFLva1q+Wknef2yfYrq6+Koi jV0uIECIMt6dr5awu4x1588UU55sLx0Hl58c/L6oM95kIYL/iR
Un7k2bHHHtvvgfYofehsKxU07z+2T7Vc1z0JRFCUPLU6AMIPPW7JWp10yVpT36NFDzGGhc7NK2kp6n7z
mNhfbtqs9pJ0X6ptfFEVRiqbFCZBa4zu+mZ1X0sDmjbrKW8/FnlNJdJi iKEpTUDMB4kcBNVfxo7D8CK
g333xTBEjo3KyCMMqKkqJUIgSsoMu6Hr40t44tWdFniqIo1dLqNRA/Cst3cFeSNNElK0qKUuTug/710
qLW1HGUEqtaXMmLJ9q/QOVpH1HQ6g1Gt6rKEqtafMCPNZUu6+Hj58aJa0t3Q9EUZRao+QCxEYFFV3y
RkRVQ+i6WSawavb18Etonw+0h9D9QBRFqTVNLkCqmX2XopJIJyjHnFSJuSot9LaS56DrORRFaUk0qwk
rjwM6VEL7geTFv2ZowV4aabP9pqLovUUURVGqoVkfSGhldZ5y6qmnNtoPJC/+NdMEyAsvvBBNnTq1Qc
mzADAvWwNmQ0ybNq1Rnyjuvh9pRakMm9TS1z7tcYt93z/u49fLateSdl5aP3zSrueT1r5/XLZ7ea9nq
fY6PrZ+Vrs+WddvrvPS6vn49WpNswQSGfkrbbaSgbgSgbhvNfcZpttos0226xBqeR6aSaySqKkSCLp
94ni0svTiIIZP/zhd6Ntt9022nTTTJMjddjttj3/eN+8etltWtL2nlp/fBJu55PWvv+eVnt5b2epdr
r+MXWz2rXL1nXb67z0ur5xa9Xc+IaQdOhYiRkuio0zWAerJNVllxySfmbpy23DuS9phmY4y222KJB+f
rrr5NW6thuu+0a1fGLESAvt++Tdr2s85TGDBkyRD6HAQMGBF8rxaLPu2lpiuf9zTffxM2qgVRKpZpLO
fhRTBTfUR+KsPKLec5J7Ybkad8n7XqVBhAo iqJUw2IpQH784x9XZE6ChQsXJv+Vjx+yW+0qdp+s9tLe
L7ofiqIoeVgsBcjb79dcURUNZFMfhqUciK48hDaaMotadfL0o+iKIps0GLMqgE0HSpF+EDWXXdd8S1
U4gN55ZVX4pdeeqns0qFDh2D7ecrEiRMLa6vSojREbfJNiz7vpqUpnvd i6wNZbbXVUq0bsvBTf0Qt1W
yN66czqdT81pb56KOPxN8zY8aMJg9VLicPPvhAwqYJr/YzIrPJ18cffxNmTJF/flsjMb9fPnll0mNM
J988kk0adIku f9PP/203ueFRvr+++9LW375+9//nqpt0wd8cG49v29+8a/d0rDP/eWXX5bvShru51Pq
ub/33nvylF577bWSi36LqEdILlYV/5lPnjxZziva0lEoiTApHJo0FVcjCL2fp6y55pry1/xAk5byayC
VUqm2RHH7CdW0VWLZ3Pjuu+/iZ599Nu7Xr1+D+9hoo43ip59+OqlVORdddJG0t+uuuwZfl80HH34YX3
LJJfHSSy/dok9EC/7617+OjQCQerNmzYpPOeWUBnUoaNPnnHNObAY3qefy1FNPxZ07d5Z6yy+/fHzTT
TfFCxYskPdmzpwZH3vssY3ao6ywwgqi7VqI1L22mvjFVdcMVgvrW9uueqqq2Iz2CUtlkeRz9vCc3j8
8cc10tHtJ2W33XaLJ0yYEC9cuFDqPfjgg3G3bt0a1TvvvPPiz77TNqj7p//Od4ww03bFCHz2fgwIE
1qzd9+vR47733bldHlt69e8fvvvuu1CuHWjxvHzSQJfjHNFw4aRoCs0czgMr/LAioe3+Xg+l4t0yyy4
rt36ZKx5EcmgG41400fhUBqdWZzbFnB/t0WNx+Qlpf/fN8fv/738tMsRJq9DHXFD0oyWyX8e00v+LL
75YZsBXXnmLrNNpKdCn119/PfrnP/8Z9enTR47Nnj07GjBgQGSEYHThhRdGJ510Uuq+MPvuu6/MOI1w

iA4++ODkaBQ98sgjkRng5PvbpUsX0RZ4ffjhh0u6fjQGIxREmxgzZkxkhENyZkPeeOMN6c04cePkGRp
hlbyTDf0688wzo+7du0eXXaZrD9oKXDfZ599tmgVjz76aNS1a1c5zv/coxmUo3PPPTd64oknIiPIo3
XXXTcaOnRotP7668tnRp0777xTPqczjgj+stf/hJdcMEF0XrrrScZK8xkRdrjmQ0bNiw68sgj5VmMH
z++sHrnn3++fM+NcBZNZPT00VJncUAWMyJAaoFpP1iQWrUgbVbvayC1nP0zU2R2kUVaH7JgBho6L09Z
3GEm3qtXL5lBvfjii8nRlosRIPGh4ad+rUKR4+fHhsBL8c//bbb+VezGQh3nHHHe0tttK6vLaFVr
BT37yk3ivvfaKn3nmmdgMRHGPHj1iM4mQNsAIrPjkk0+Wz9cM8KKtoFmMnTpV3gc0uT/96U/xpptuGu
+3337xY489JrNRrjl48GDpZxr4C3nmnPvXv/410dpyQL0//fbbRSs1E4r40ksvjY2gjHfaaSex9b/22
muied1xxx3x0uusE5sB0zmzDiNY5N6MAJfnYoS8rM/i f5g2bZo8f/sb2nnnneM//vGPhdabPHlybCYf
8YEHHihjB/3ZZ599RNt755135LyWSrP4QGq1T0UtNYu8mGea/FeaFm3TbIGgdVWqeTUH//jHPyRNzsS
JE6NBgwaJxmB9aISRz5s3T+z0M2f0FB/GAw88EP3qV7+SGSsa+YQJE0TT6NmzZzR27NjIDOKitaLBmE
lK/WZhaLXs6b/lllvKa+z/zGTx8+2www6Sjgc7/1tvvSXayoMPPhtvuvu0a233ho9/vjjMus2A2t0w
w03NEqNgbyZcOBAscFfccUV0l5Lg+zUP//5z6WfPE8jGEVLQuMywjnaYIMNxFLBPR9wwAHRYJEjZZyw
xUxGxCeBNsL9Y63AuJFixIioW7dusqob7dcIg8gM8PIId/OKLLwqtx+eNhrnFFluIfxQLBr4xNBM0pf3
3318+ixZLnSwpHpo0ldAK7CIK0jt0vabUQNq3by9//XtkFuTin2dL1rNJU8c8ZXFn0qRJ8S677BLvsc
cesfLBJUdbHmPGjBFNoW/fvvGrr76aHC2NERwyQ2bmPGrUqLhPnz7ymW2yySaicW299dbxWmutFZvBU
0z4ZuCX2XUazL7xE15//fWiYdx2220ySz/99NPFLm9B60ED0v744xv4X4yAE01mLVVWER+B78NrKfAM
jjjiiNgMvPHDDz8sx+gr/qi11147PvHEE0VLszBjRrMzgjae03dufn9998nv6uyzz5bjRhjJc+fZXXz
xxfX3jSaGhsNnOm7cuELr4f8IMWfOnLh//7xeutF999993J0ZYFz7PJBuHtlyBsvHGG8ft2rVL3i
0PHJBuW2mFH2QWedtqS/DcU09xpt94442xmVnHZrYtJom//e1vMjiYWvxSu/ngh4SZwszS5XPERISQo
++Y2/iL85l6RhuIn3vuufj5558X09Sdd94pDt9Vv11VHKscQ0AY7U0+m127dhXhstJKK8VG84hXW221
2Gg0Mgjy3eY6tEwo+EMPPSSDFoIG05M19fGsuMbqq68uAxn05WHDhonw4LoIGvs7oY+33HJL3KVLl/i
YY46pd/S2NPjch330URHWhPUjJDEHYWrjOdJ/M3uPjQYLJj++KwzWfBZ8Vscdd5wEPRjtpH4Q57lsv/
328lwYtJm0/053v4vXWGMNEQKjR4+W6xZdz2gr0gc+R/p/7733St+ZkGLOevPNN6V/LY02IUD8wdt/n
x9mqF4e/LbSSp62Q+eFSiX9XFxhgCi1Zgab9ueff57Ubj6mTJki2mOojxQb6cQA50fkUBhg7rnnnvjL
L79MwMwMwGJ177rrruRI+Pl07NgxHjRokAycFgYphMhBBx3UoC6DL9dLIAD+4lNA08WfhzBrydBfhAF
amntfyyyyzjPhCrPaBL4FoJrcOmhyTEldAEq3FoI/vya2L5ku01/fff194PXxUI0aMaPA+Zf311xcfla
effipttUR4/jWLwtp5552T/5oX7Mqu32XPPfdsY08luoUoCGzN5fnp/LbS8PsQosi2FEVRag3jVc0Ei
KIoiTJ6QYDokmhFURSLiLSAKIqiKBWhAkRRFEWpCBUgiqIoSkWoAFEURVEqQgWiOiikuHeswU7+VRRF
UZR8IDra/ec/le8RriiKorRNUD7aLVjwb8lGqSiKoih5QGaIAPnuu+8k5TBphRVFURSLFMgKZMbXX8+
P/h/2RoQk7TZ4uwAAAABJRU5ErkJggg==",

```
24     }  
25  
26         const res = await  
this.nMPrintSocket.DrawLableImage(DrawLableImageParam);  
27     if (res.resultAck.result != 0) {  
28         return;  
29     }  
30     //进行下一步操作,继续绘制或提交  
31 }
```

3.8 标签预览 generateImagePreviewImage

代码块

```
1 export default class NMPrintSocket {  
2     /**
```



```

3      * 生成图像预览图像。
4      *
5      * @param {number} displayScale - 图像显示比例，表示 1mm 的点数，可调整预览图大小。
6      *                                     例如，200dpi 的打印机可设置为 8，300dpi 的打印机
可设置为 11.81。
7      *
8      * @return {Promise} 返回一个 Promise，解析为生成图像预览图像的结果
9      *
10     * @description
11     * 增加方法说明：
12     * 1. 在调用此函数之前，必须确保图像数据已准备好，否则无法生成预览。
13     */
14     generateImagePreviewImage(displayScale)
15 }

```

代码块

```

1 //返回数据示例
2 {
3     "apiName": "generateImagePreviewImage",
4     "resultAck": {
5         "errorCode": 0,
6         "info": "{\n\t\"ImageData\" :
\\\"iVBORw0KGgoAAAANSUHEUgAAAZAAAADwCAIAAAChXqV1AAAGAELEQVR4AezBeaznd33f++fr8/2db
WbOeMYzY+Mzb+DxgAk2lN5QGivRC71SkyqtKhVVAZEAOUsQ6U1RK4SaRKJqdRM1EamKyI3C4pKNViit
qLQJ/aMNNA2JaEoANxhsg5fx2B4vs585y+/7eV7zTY80R/POVUa3FhrpPB5R2bVr167rQVR27dq163o
QlV27du26HkRl165du64HUdm1a9eu60FUdu3atet6EJVdu3btuh5EZdeuXbuuB1HZtWvXrutBVHbt2r
XrehCVXbt27boeRGXXr127rgdR2bVr167rQVR27dq163oQlV27du26HkRl165du64HUbKWSaioVJJQU
akkoaJSSUJFpZKEikolCRWShIqKpUkVFQqSaioVJJQUakkoaJSSUJFpZKEikolCRWShIqKpUkVFQq
SaioVJJQUakkoaJSSUJFpZKEikolCRWShIqKpUkVFQqUbKWSaioVJJQUakkoaJSSUJFpZKEikolCRW
VShIqKpUkVFQqSaioVJJQUakkoaJSSUJFpZKEikolCRWShIqKpUkVFQqSaioVJJQUakkoaJSSUJFpZ
KEikolCRWShIqKpUkVFQqUbKWSaioVJJQUakkoaJSSUJFpZKEikolCRWShIqKpUkVFQqSaioVJJQU
akkoaJSSUJFpZKEikolCRWShIqKpUkVFQqSaioVJJQUakkoaJSSUJFpZKEikolCRWShIqKpUkVFQq
UbKWSaioVJJQUakkoaJSSUJFpZKEikolCRWShIqKpUkVFQqSaioVJJQUakkoaJSSUJFpZKEikolCRW
VShIqKpUkVFQqSaioVJJQUakkoaJSSUJFpZKEikolCRWShIqKpUkVFQqUbKWSaioVJJQUakkoaJSSU
JFpZKEikolCRWShIqKpUkVFQqSaioVJJQUakkoaJSSUJFpZKEikolCRWShIqKpUkVFQqSaioVJJQU
akkoaJSSUJFpZKEikolCRWShIqKpUkVFQqUbKWSaioVJJQUakkoaJSSUJFpZKEikolCRWShIqKpUk
VFQqSaioVJJQUakkoaJSSUJFpZKEikolCRWShIqKpUkVFQqSaioVJJQUakkoaJSSUJFpZKEikolCRW
VShIqKpUkVFQqUbKWSaioVJJQUakkoaJSSUJFpZKEikolCRWShIqKpUkVFQqSaioVJJQUakkoaJSSU
JFpZKEikolCRWShIqKpUkVFQqSaioVJJQUakkoaJSSUJFpZKEikolCRWShIqKpUkVFQqUbKWSaioV
JJQUakkoaJSSUJFpZKEikolCRWShIqKpUkVFQqSaioVJJQUakkoaJSSUJFpZKEikolCRWShIqKpUk
VFQqSaioVJJQUakkoaJSSUJFpZKEikolCRWShIqKpUkVFQqUbKWSaioVJJQUakkoaJSSUJFpZKEiko
lCRWShIqKpUkVFQqSaioVJJQUakkoaJSSUJFpZKEikolCRWShIqKpUkVFQqSaioVJJQUakkoaJSSU
JFpZKEikolCRWShIqKpUkVFQqUbKWSaioVJJQUakkoaJSSUJFpZKEikolCRWShIqKpUkVFQqSahcv
rwxn2/05/0trS1Aba0BR44cofLCC88lCzCXEVsyQDftxgMHQzW9e5ZJEjUT9YYbbqBy9uzZJCqQBEgC
7N+/n8rZs2eBJCqQhMkNN9xA5dy5c733JGomTg4ePEjl7JnTI7MkdIHIONNx8ciNq1See+45rqAC6k0

```

33URFpZKEiKolCRWVShIqKpUkVFQqSaioVJJQUakkoaJSicq1SEJFpZKEiKolCRWVShIqKpUkVFQqSa
ioVJJQUakkoaKy0wgb5y/svWE/lbNnz6pMMmFyww03ULL8+TLQJkDvfRgGYDabUVHZpgJJ1NYaFRVQg
SRM1NYaFZVKEioqoAJJABVorVGZz+dJeu/qMAx711dXFykcvHiRcBtgAocOHCAygsvvKDyoiwwdtPN
XL3p8M1UVCpJqKhUklBRqSShoLJJQkWlkoSKSiUJFZVKVK5FEioqLSRUVCpJqKhUklBRqSShoLJJQkW
lkoSKSiUJFXXtt3/n3GvvWZ4Nw8rSrA2ttWEYFhcXqaiACiTpbfWbHnvssTvvvJ0KyjY1CZMkV0bzOd
B7d9J7d7Jv3z4qzz33nBPAbb33Y8eOUXn44YeB3ruTPlhvvfdeKp/730d67/P5XJ3P530yn8//5t/8m
1Tuv//+PmGnd7/73VTuv//+J0revXs3Njb0nj27urp65MiR7//+76cyn8+dJFHn83lrTV1ZWaFy8uRJ
4NixY+yUhIpKJQkVlUoSKiqVJFRUKkmoqFSSUFGpROVaJKGiUklCRaWShIpKJQkVlUoSKiqVJFRUKm
oqFSSULL48WISFWitDcOQycLCApXe++bmprq0tJTkyx/7RJLz58//w3/4f1E5fPjw8vLyysrKMAyttD
47kwcffJDKK17xKnUcR7eN49h7P336NjXl5WV3SgLM53MqS0tLTDJprQGttQsXLlC56aabgDYBwmtJg
CeeeILKPffcA7TWkrTWkrTWkvYp//E/qHz3d393ay3J/v37V1dXDx06tLq62lr7Z//sn1E5d+5c7z3J
k08+eddddy0uLiZRW2tUVCbq0I7qwsICkISKSiUJFZVKEioqLSRUVCpJqKhUklBRqUTlWiShoLJJQkW
lkoSKSiUJFZVKEioqLSRUVCpJqKhUklBZW1tLwrZhGJIACwsLV066666tra09e/ZcuHDh4sWLW0Li4s
LCwqlTp6jccsst6sbGxnw+T6ImAc6fP0/l80HDSVprSVprSYAkJ0+epHLnnXcmAdQkgAo8+uijVG655
Rag9+50L7zwApXV1VU1CZAESAKc03e0yk033dRaA1prucLjjz905fjx40CSzYmaBHjqqaetzGazD37w
g+9973s/85nPf0ELX3jta1+7Z8+eo0ePvvGNb6SiUklCRaWShIpKJQkVlUoSKiqVJFRUKkmoqFSici2
SUFgpJKGiUklCRaWShIpKJQkVlUoSKiqVJFRUKkmorK2tqcMwJPnYxz721/7aDxw7dkubULnnvvuA3v
uLS5fw19dXV1cXfXfPnj178uRJKi972cvW19fHcWytAa01JmfOnKFy6NAHJioTJ2fPnqWyD+9eoPf0t
iTA2toaLU0HDjFprWUCJHn66aep3HbbbUycAE6eeuopKocPH3bCNhU4c+YmLc0HDzNRmahJnn/+eSor
KytqEiAJkMnFixepqFSSUFGpJKGiUklCRaWShIpKJQkVlUoSKiqVqFyLJFRUKkmoqFSSUFGpJKGiUkl
CRaWShIpKJQkVlUoSKr3397//T/5kz+5urr62c9+9i/+xb/4oz/6oydPnvxP/+k/Ufme7/me3vvGxs
a5c+c2Njb27Nkzm81673/8x39MZxV1dXfXcRgGtqnAs88+S2X//v1uA1Qma2trVFZXV4EkQBK2nTt3j
srq6iqgsi2JevHiRSr79u1jkoQrXLhwgcr+/fu5QhI1ybLz56gc0HAAS0IEUIHz589T2bdvH5WLFy9S
+ehHP/pbv/VbP//zP3/rRbc6aa0BSaioVJJQUakkoaJSSUJFpZKEiKolCRWVSLR2ffvM5/Nnn30WWF5
eHobhN3/zN7/85S//9E//NPBHf/Rht99++1/6S3/p0KFD8/l8bW3t3LlzeybDMLBTEvXy5cvnzp1bXl
4+c0DAMaXMKD3vrm5ef78+eeff35tbU2dzWaAylVUrKUSkip/OjUJExViojJJoiZhogJJADUJ0yXpv
Q0ZqExUtrXWVP4UKtuSACqTJGoSIAnghEkSKkm4SpLe+1/4C3/hM5/5zEc/+tHXve513/zmN7/whS98
6EMfYqImOXpzmMGDB9m1U1R2ffucevIJMsxms49//00PP/74Rz7yka985Sv33nnvF77whbe//e2z2cz
J1tbWpUuXxnFcWkVbXnFUx3HsvavjOKrz+Xwcx957EhVQgd67ylWScIUkKjsLUYEkbF0BJPyZqUmAJC
oTNQmgAkkAlUkSlaskUZMwUZPwZ5CEbWoStqmtNa6gJvEKgNpaG4YhiQqoSZgk4SpJuEISrqCurq4+9
NBDn/rUp17/+v/j137tV37u5360yebm5rLz544c0cKuK0RL17fJxceedfGEpFX3z4x//+Hvf+9719fXb
b7/92LFjFy9e3Lnnzxve8IZ77713ZWVlc3Pz6aefPnnY5NGjRw8fPgyovXegT+bzee/9/Pnzjz32mHr
s2LG9e/eq8/m89z50LL68+PDDdz/55JPr6+tAktaayv8mKpCE/9+SMFGTqEn4M30SR0UKKldQk6hAEp
U/gyTDMLTWAJUrJOEKsDgpCTsLUVtrN91006c//RsPP/zw3Xff/bWvffVv/+2/3Xs/c+bMoU0H2HWFq
Oz6Nnn69DMf+Cf/5BW33/GTP/mTm5ub//Sf/tOf+ZmfAXrsv9ksyd69e0+cOPGGN7xhfX19YWHh7rvv
XlLZUcdxVMeJurW1tbm5eerUqYceeuJqoU033nprknEyn883J88999wDDzzwzDPPz0dzJklms1lrLQm
QBEGCJGGShCskYVvvPQmg8qdQkzBRgSRMVLylUZMwUZ0oSdimJlGTsE3lKiQGAr13tjkbNAAq21T+bI
ZhaK0BKldIwiQJOyVhpyRM1EyAX/zFX/qDP/iDf/SP3veLL33pTW960/r6+qFDh9h1hajsjz57z/4s
ZuP3PQTP/ETJ0+e/OAHP3j//fcn4QqLi4v33Xffn//zf/7y5ctHjx59+ctf3lpzMo5j730cx967ur6+
/sgjjzzxxB033377LbfcMgx730cxz722tp66qmnPve5z506dWocRxVora2uri4sLayT2WzWJsAwDG2
ShEkSrqaCKhOvidp7T7K5ufmNb3yj9/7hD3/4X/7LfwN03t/+9rf/1E/91Gw20378+Pr6+g/90A998I
Mf7L2rvfckvXeVyU033QR8+MMf/lT/62+N46g++OCD3/M93z0bzb2ts6fP7+1tfWbv/mbP/IjP9J7T
6JyFTUJoCZhJ5WrqOyUHG2tNaC1xk5JuEISKkm4QhImSZh853d+55vf/H++5S1v0X366XvvvffAgQPs
ukJUdn2b/I2/8Tc+9KEP9d6PHz++sLCgzmazYRj6ZBzHG2+88S1vecuJEyfw19df8YpXHD58uLWmbk2
SAPPJ2traV7/61VOnTt1zzz033nhj732+bWtra2Nj47HHhvut3/qt06dPj+PIZBiG1dXVYRjaTklaa9
kGtNaYJAGSqICapPeuMLGTjOMIOPmxH/uxH//xH++T97//Z/61KeAcRyTPP7448Dtt99+6dKlhYWF+
Xzut67muTf/Jt/873f+73PPPPMa1zmmefbb3nuTo0aNPpfVU7/3jH//4Bz7wAZVtSXgptdYy4SpJ

2JaEnZJwLSRsywR417vedeedr3jPe97zu7/7X9/85jex6wpR2fVt8ta3vvXXfvXXzl84//f//t//t//
23/beh2EAnABJhmFQgTYBVEBlogJ0gGEYXvayl33Xd33X3XffvbCw0Ht370vr6w9+/Wu/8zu/c/78eZ
XJ0tLSn/tzf251dXV5eXlpaWlLZWVxcXfHYaFNlpeXW2uz2ay1NgxDmyRprSUBMGsSqExUJr13t12+f
Ln3nqRPtra25p0NjY3FxcWFhQUmSba2tlprW1tbrbV//+//fZK//Jf/8uHDhy9duvTrv/7r4ziqb5os
LS0Nw/Af/+N//MxnPu0ktQaoSXgJJGHSWkvCFZJQSQIKoZKESRImsdT19fW/83f+3r/4Fz/TwtuzZw+
7rhCVXd8m7/nRv/eRj/zixUsX/+pf/au//u/r3KNVP50SdimcgU1CX9mSVT+PyWh8h/+w39YXV2955
57Tpw48cADD3z4wx9+3/ve9+lPf/rnf/7nH374YfWFF1648cYbl5aW1tbWkgCbm5uXL19+y1ve8slPf
vJlL3vZuXPnNjc3jxw5Apw+ffqmm24CXv0a1/zxH/8xf4ok/G+ShG25AlDjwrYkXCUCJkEQFkjBJwhV+
6Zd+6U1vetPBgzcuLMzYdYWoXIskVFQqSaioVJJQUakkoaJSSUJFpZKEikolCRWVq7ztB9/6a5/6dSr
DMKhsU5moVJJQUakkUblGSdhJpZKESZLjx49/9rOfnc/n3/Ed33H+/Hkqv/qrv/rII4/83b/7d48ePX
r58uWf+7mfe8973nPw4EGVvyLy8ubmpsp0KpXWGPXe05XWGPmKXGECrypLS0tJgCRsS7K2tkZl7969Q
BIgCZMK58+f56Wkcj2LyrVIQkwlkoSKSiUJFZVKEioqlSRUVCpJqKhUklBRucp3fcdcbP//5P6Aym81U
rjK0I5XWmspVVCpJ1CTspFJJwhXUJIBKJQmQpPd+80DBT37yk//u3/273/7t3z516hSVJIAK/Jf/8l/
uvPP0j3zkIz/7sz+rUvnEJz7xrne9i6uoVFprVHrvVFprbEsCJAhm8zmVLZUVJkmYJFHx1taorK6uMk
nCFc6fP89LSeV6FpVrkYKSkiUJFZVKEioqlSRUVCpJqKhUklBRqSShonKVRx555Pjx41QWFxd770xUJ
mravnUoSKiqVJFRUKkmoqFSSUFGptNao9N6pDMNAZRxHKrPZDFCTAGoSYD6fU5nNZlWlydbWFpWVLZUK
QBImSYCLFy9S2b9/P5MkQBIgyZkzZ3gpqVzPonItklBRqSShoLJJQkwlkoSKSiUJFZVKEioqlSRUVK7
y1a9+9dWvfjWVpaUlwG1sm8/nVFprKldRqSShoLJJQkwlkoSKSiUJFZVKA41K753KMAxUxnGkMpvN2J
YEyGRjY4PKvn37mCRhkgQ4f/48lQMHDiRhogKbm5snT5688cYbeSmpXM+ici2SUFgpJKGiUklCRaWSh
IpKJQkvLUoSKiqVJFRUdprP5w899NCrX/1qKsMwLCwstNbU3rsTYGtri0prjSuoTFQqSaioVFprVHrv
VFprVHrvVFprVHrvVFprVHrvVIZhYfsSts3ncyoLCwtMkjBJAmxsbFBZXV1lkgmQyZkzZ6gc0nQIyLZ
Pf0IT3//933/+PkbbrIBl5LK9Swq1yIJFZVKEioqlSRUVCpJqKhUklBRqSShoLJJQkvlp7W1tccff/
yee+6h8vu//vnz5//vu/7vuXLZWAYhiSttTNnzLDZv3//OI7z+RxwW59QyYSr9N6ptNao9N6pDMNAZ
RxHKsMwUBnHkcpSNqMyn8+pLCwssFMSYHNzk8ri4iKTTIBM1tbWqBw4cABIAMTC5Pnnn6dy880303n6
6adbaxsbG48//vgf/uEf/uAP/iAvJZXrWVSuRRIqKpUkVFQqSaioVJJQUakkoaJSSUJFpZKEisp0Fy5
cePrpp0+c0EHLda973ec//nL5eX/9t/+25kzZ376p3/661//epLTp09TOXDgQ099Pp8DSZiM47i+vk
6ltUal905lGAaukmQ+n10ZzWZU5vM5lYWFBSpBW1tUFhcXqWxubLJZWlpiWxK2ra+vU1lZWQFaa2oma
mvtwoULVA4d0sSkTZZEzeSZZ56hogKnTp3at2/ft/zET7z1rW9dXFwchuF1r3sdLyWV61lUrKUSKiQV
JFRUKkmoqFSSUFGpJKGiUklCRaWShIrKTmcmd911F5UHHnggSe/9fe9734c//OETJ0586EMfetvb3nb
zzTdTueGGG9RxHNUkbltfX6cyDANXSMJkPp9Tmc1mSbjK1tYwLYWFBbYLYdvm5iaVxcVFdkoCbGxsUF
leXqayvr50Zc+ePeyUBLh06RKVffv2AUmYZNvZs2ep3HzzzUySAK01JqdOnaLysz/7s1/60pf+wT/48
fl8M0lrbTabtdZe97rX8VJSuZ5F5VokoaJSSUJFpZKEikolCRWVShIqKpUkVFQqSaio7HT690nLly/f
eedVB566CFgHMckvffWwu99Pp/fe++9VG688cbW2ji0vXc1iZMLFy5Qmc1mQBJ22traorKwsJAEULn
C1tYwLcXFRsqbm5tUlpawQgxsBFBZWVnhKknW1tao7Nu3TwWSAEmYXLhwgcr+/fuZZAJk8sILL1A5ev
RoJkBrLcmRI0f+83/+z/v376fy3//7fx/HMVdpk9lstrC0eNfLX8GunaJyLZJQUakkoaJSSUJFpZKEi
kolCRWVShIqKpUkVFR20nXq1Hw+v+0006g88cQTKtB7d5JkHMFjx49TUR977LEPfoADV/ALv6DecMMN
wIKtJx5++GEqS0tLTpJwhc3NTSrLy8uAyrYkwPr60pWlpSW1tQaobNvY2KCytLTEJALXWF9fp7KyspK
EKyQBLl26RGVldZWdkgDnz5+ncvDgQSYq0FrL5LnnnqPyR3/0R6997WvVz3/+869+9asffvjh97//A/
P55uc+9zkqf/iHf8i21lomrbUkwzAACwsLd911F7t2isq1SEJFpZKEikolCRWVShIqKpUkVFQqSaioV
JJQUdnpiccf+644w4qTz31lCaRiZpEPXr0KJVTp04NwwDM5/NbbrkF+0t//a+/0Uv/1f/6l9R+Yef
+IHW2jAMKpPWwpJPf/rTVN72trcBrTWgbUvy0Y9+lMpP/dRPzWazhckwDLPZbGFhYTabvfvd76bye7/
3e6212Wy2sLAwmywsLAZDcMdd1CZz+cqExXovasrKytUvvGNb4wTtU9U4L777qPy4IMPttaGYdja2h
only9fvnDhwL/5K3+Fyjvf+c5//I//8cte9rKTJ0/OZjNga2sryX333Ufli1/8oportNaStNaGYQCSH
D9+nF07ReVaJkGiUklCRaWShIpKJQkvLUoSKiqVJFRUKkmoq0z0jW98Y2Fx+fbbjLE5c+YMNHUYht57
a20Yoq6urLJ5/vnnx3EExnHsvQNqa+3o0aUnnnvuXEc6iZqEmOHDLC5fTp05moXOHikSNUnnvuOSY
qoCYBjhw5QuWZZ55JAiRRgdaaeuTIESqnT59moiZhkuTIkSNUnn32WZUrJJn57fccguVp59+urUGjO
PoB0i933bbbVS+9rWvJVEBJ0mAV73qVVS+8pWvMFGTtNaybRgGQL377rvZtVNUrKUSKiQVJFRUKkmoq

FSSUFGpJKGiUklCRaWShIrKTg899NDKysptt91G5eLapSFNTTIMg8pkaWmJytrausrEbUlWV/dSef75
55kkUZ0owKFDh6g899xzSQAnQBLg80HDVE6fPp1ETQKoQJIjR45QefbZZ1UmKpMkN910E5Wnn35aBZK
wTb3llluoPPPM2rvPQmgAkluueUWKqdOnVKZJ0m9M7n11lupPP7440nUJICTJLffffjuVr3/960nGcQ
RUQE0yDENr5EXw8lccZ9d0UbkWSaioVJJQUakkoaJSSUJFpZKEikoLCRWVShIqKjt985uPLC0uHrv1d
irjxlprDbAFUJMAbVii0t2MqICaDE5mC0tU5lsbahIVaKICs+UVKhvra6213ruaBFBba4tLK1TW19eZ
qEASJ3v27KGytramMLGBJMDexXupXLp0SWXSwgNUY0/evVQuXboEDMPQewd676213vu+ffuoXLhwQeV
bGqAmAfbv30flZJkzKt/SAJXJoUMHqTz66KPQ/4QTIJCKtZYEUPPl7Frp6hciyRUVCPjQKhUklBRqS
ShoLJJQkWLkoSKSiUJFZVKEioq0z36zUcWfXeP3Xo7lf6pWEYSaXJ+BN50QIVFToTNQmTZKCiI47j6D
AMJNDtXR1my1Tsm4CaRGXsmQ1A2iIV++Y4jq21+XzewhuGofcODLNlKv0ty621tGbvahKg9z5bWKEy3
7qcbpIeBtJ7twWYLaxQcWtdZZJEBdRhaQ+VvnLZBZIAKpNhaQ+VjFWLTdR5pAuk0+K+1YNUTj35hNon
6ji0vXcgSWsNaK3dcecr2LVTVK5FEioqlSRUVCpJqKhUklBRqSShoLJJQkWLkoSKyK6PfvORxcXFY7f
eTmW+fsmWWRoJLbwoAZIFKn7LmERNBuia2tqmim7htyQhsfckahuWqPRxo/feWlN778MwRDo0s2Uq43
y9tab23ofZrI9jEnWYLV0Zb10G1GEYeu9JAHW2sEKljxtM1CRqXtTNwjKVvnkZSNJ7b8Ng70za4gqVc
WNNba2pQ0+doTUZlvZQ2Vy7AKjAiC9Kou7dd4DKU6d0qr33cRx77+o4jsDdJ17FS0nlehaVa5GEikoL
CRWVShIqKpUkVFQqSaioVJJQUakkoaKy08lHH2uNY3fcSWW+finbbAGSkBctUNFRBZJwhWSgMo5bLfb
eW2tA7+RFRSUDLXG+3lpTe+9sSzLMlqn0cQNQW2skfRyBJllypjLO15vYorbWeu9qe9GwRMwt9R4a6d
hao4sCWVimMs7Xm6hJfFFLE7Utrldpm5fVJCMmcTKQYwkPLY31i036i8KLVKD3vm/1IJVTTz4Bj0PYe
1d772rv/e4Tr+KlPHI9i8q1SEJFpZKEikoLCRWVShIqKpUkVFQqSaioVJJQUdnp0Ue/sbgwHLv1Tirz
9UvZxtD4E3nRAHudVbYLAXrvw7BAxT4Heu+JfEvTUZ0trFDp44baex+GQWwSpA1LVmb50mNvwzDvY+W
39EQAACAAASURBVJJhNptvbSWZLaxQGefrQJLee17UZdIWV6g43+jjCLTWSIB5H2dtyGyJilvrKttUJs
PSHip98zKgAgGSEQfSFleobF2+mKT3zmRu70h33+pBKqeeFELtvY/jqPbe1d77iVfew0tJ5XoWlWuRh
IpKJQkvLUoSKiqVJFRUKkmoqFSSUFGpJKGist0j33xkcXHx2K23Uxk31lprvCgxvCiJ2oYlKjpyJb8F
aMMCFfscOjCOY7apbViiMs7XkwBqEibqMFumMt+6nKG10RHbbHDSQJI2LFEZ5+tAa42uvfeQri2zhRU
q863LTdTWmppEBdriCpWtzTVgEJIRgXSTDEt7qIwba6213nsStfc+m81678PSHirrly8ATXpQASd79x
2gcurJJ9Q+GcfRCXD87lfyUlK5nkXlWiShoLJJQkWLkoSKSiUJFZVKEioqlSRUVCpJqKjs9Nij31hcX
Dx67DYq48Ya0IahYyZqXtQWqfQ+B5JARw1hUFubUenjhtpaG8cRSKImGwBLVOZb14fZzN7Z5mS2sEJl
vnW5SQ9N1MyGdHsYZstU5luXgYEAPaSBRG2LK1Tsm31rztB670mAdJO0xRUqmxuXmgzD0HtPgsg3DEt
7qGxdvpgESKICakuG5b1Uti5fHBFQARVQ9+47Q0WpUyf7VZICv/uVvJRURmdRuRZJqKhUklBRqSShoL
JJQkWLkoSKSiUJFZVKEioqV1Aff+ybi4uzo8fuONI3LwdIenhRa03Ni9oils9/6YskaBIgE+A7XnmFl
f/5wJdba2prTU3Se09yz6tfQ+XrX/uqmqT3nonaWrV7xKuoPPT1B9UkY+9J0NYacPeJV1F5+KGvAa21
3nsSJ0m03/1KKg99/cEkTNRmgLuOn6Dy0NcFHIzhPo6BJL33YTaz9+N3v5LKNx55CGitAWprLZPbbr+
TysULZ1prQBIVUIG9+w5Q0fXkE2rvfRzH3rvaewfuPvEqXkoq170oXIskVFQqSaioVJJQUakkoaJSSU
JFpZKEikoLCRWVK1y6dGnv3r2nTp06duwYlXFjrbUG2AIkUV944YXDR26h8sY3vrG1NgzDfD5XoUNL8
vnPf57Kd37ndwIq4KT3rn7pS1+icu+997bwGHEcl5eX19fXk6gPPPAalde/vUbGxtqktaaCiR+5Sv/
k8prXvMaLUkSYDabbWxsFPWrX6Xy+te/Xh0nS0tLTpJ88YtfpPLa174WUJmoSYAvf/nLVN74xjf03od
h0Lyo/S989rP/lcr3fu93t9bUcRxba+rW1tab3/zmf/7P/28qJ594LMk4jn2bmuT43a/kpaRyPYvKtU
hCRaWShIpKJQkvLUoSKiqVJFRUKkmoqFSSUFG5wtbW5l0nnlxYWDh67DYq8/VLwzD0kERtw4DpvQ+zR
Sr33/9xJmPf6r0PbQFoA+/44XdT+djHfmk2DLPZrE+S9N6T/PA73kXllz95P+BkNpvN5/Pee2vth9/x
Lir/+v6PA22ytbwVRG2t/dAPv5PKJz7+0WEYWmu99yQqk0TtP/QOKr/yy/+6984kiQr03t/xzh+h8iu
//K9ba1tbW7PZbBxHoLU2n8/f8c4fofLLn7wf6L0nGccxZCN3vPNHqPzi//OR2WwGbG5u/sZv/Mb6xk
ZrbRzH3/3d36Ny6skngPl8Po6jk967euKV9/BSUrmeReVaJKGiUklCRaWShIpKJQkvLUoSKiqVJFRUK
kmoqFzh61//2r69e5IcPXyblXFjLRNb2JYXtUUq4zja5733+Xw+jipjfbZh1tr+G49QuXj2eRVQW2sB
EmDvDTdSuXTuBTWJmgRIMo7j6sHDVC6efT4JV1CBfQcOUbl49nmVSRKVyerBw1QunHkuyTiOrTVABZK
sHjxM5fwLz6pA77211ntPAhw4fDOVF04/LURNogIqc0jmo1TOPPt0EqC1No4jMJvNhmHYs/8glSDPPq

723sdxVHvvToZhANSL2dKtd97Brp2ici2SUFgpJKGiUklCRaWShIpKJQkVLUoSKiqVJFRUrvD8889eu
nRpZXn5pptvoTJurLXWAFuAJEzSFqn0cSutofwnQRNW6TSx40kQ0+9tTafz4dhUIfZMpX51uUkgNok
yYjAbGGFyrrixBqiZz00DsWW2sEJlvnUZGIgKqK01IAvLVPrWmpqEidpaAzJbodI3LzPpvScBWgJkcYX
KuLEGqELUJrYsL06hsrL2AVBHfBHgt2R1/wEqp558om8bxxHovQNJWmvqvwLibcfuYNd0UbkwSaioVJ
JQ8f9tD/5ibT0L07//vs+79t7nj41tbIxtQpikabAh7QUNMGkhqtSZqGqVaJRqRp00hokmE1VVe1Gph
Y4qFUGmTG/SXuRmmuk0N71opVZRLiJ1rqYaLJIATjMNJIMZwPgPAYIdjPH5u9f7fLv80g897n7Z56zT
/dK9lp/PRzMHYBw1c4DMUTMHYBw1c4DMUTMHYBw1t3jhhW9dvfLywcHRY297e+aMN66WUGKSDSCJpHC
Y0XW8ASQLYF0nAZJQDjNnXF8vpWRjrAHJHjqsLmR0vXkNqOQ1rsdhGGqtW9GLzL1548pqtVqv16WUJL
VWNqqrC5czZ339yJAM6jiONEnK4cXMqTevAev1GkgCqGXj8GLm1JvXMLGBJECtdTi6lDnjjauV1LiSW
mspRQWGo0uZc+3qy5moSWqtmDxz7wOZ8/xzzySptY7jWGt1koRJKsuXL7/1kcFsvR5qtgFkjpo5Q0ao
mQNKjpo5Q0aomQNKjpo5Q0aomQNKjppbfP1Pn1/fGFfD8Njb3p45442rpZRsgATiHhKY0dabNQKpJgG
cDKsLmbM+vpYEyOsNqwuZsz6+BqhAMd/DwYXMGW9cTcJGKeN6PQxDJerq4GLmrI+vARnrGEspVDeA4e
hS5tSb14CxJFWqQCbl8GLmrK9fATJRK9kopawOLmb0+voVLuKsJ8DqwuXMuXb15TR0aq1J7n3TmzPn+
eeeSVIbJ7VWoJSS5PI99771rY+kez3UBAPIHDVzgMxRMwfIHDVzgMxRMwfIHDVzgMxRMwfIHDW3e065
Z4ZSgMfe9vbMWV+/UkoBAhJAZaMcZo4eJ0VNohZQ2SirzBnX150UYK2shlqrmrR1cDFzxhtXVSZjZKN
aycHhpcw5vnkVSFLMRq21lMLGwYXMWV+/ApRSnQZYZGrC5czZ7x+BTABxjgENclwdClz1tevAEnUTI
Akw9GLzFlfv5KmErUYHhXhcuZcv/ZdNR0qY9xIcs+9D2T0155/Vh3Hsdaq1lrVJEyS3Hvfg2956M3pX
g812wAyR80cIHPUzAEyR80cIHPUzAEyR80cIHPUzAEyR80tvvLVL1840KSUxx77ocwZb1wFklCKZANI
QjnMH0vaBIzmVUVHNSph5ozr68VYGMdxRVljCcVwcCFzPL6ujrGUQrXWWkpRh6NLmTPeuJoEqMRJMRu
rC5czZ7xxVR1KqTpGNqobBxfvyZz19StJABVioiZZXbic0eONqyqgMqm1AsPRpcxZX78CqJmoSYDVhc
uZc+3qy5k4SaLUWt903wOZ8/xzzyQZx7E2ahKglKLe/8D9Dz74cLrXQ802gMxRMwfIHDVzgMxRMwfIH
DVzgMxRMwfIHDVzgMxRc4uvPv3lg4NhWB0++ujbMufmjStA2TCVMElC0cwc680aCyuV1EC1Yspwldl1
vKEmqbUySQKU4Shz6s1rSSwU40aBapJyeDFzxhtXcwsVSDIcXcqc8cbVJGoppdYKZDIcXcqcmezuDGH
DCeDk40I9mb0+fiUJoGZSSLGH00uZc3ztFSCJClTys0jy5lz7erLSWwyqTVvuu+BzPna88+q6/Varb
U6qbWWSa31zQ8990YHHkz3eqjZBpA5auYAmamDpA5auYAmamDpA5auYAmamDpA5am7x3NNfYZVy4
ehtD/9Q5ow3rpZSKgGSAJlQDjPHutaRUqKZ1FpLKZTDzBnX14EkGGSJCajl8GLmeHw9G3A8roEkQ3jV
wYXMWV+/UkqptWYC1FqHYSiHFzNnvHG1ErUYNpJaa1bD6uBi5ty4/gqQZgi1VmB14XLmjDeulK06wi
k4FiHsFEOL2b0jeuvZDKESkq1FopZXbic0deuvpxEkuprktRa33Tfg5nzteefrbdiUmvNBEjy8EMP33
v//eLeDzXdD9xXn/7ywcGwWh2uDodSykAZNgKrYQhDKUkoJZBEAiSREJKSV9VsaBpJqmmATGqtmVctZ
EMtG+bPDSVjVTMUCTAer7/zne889dS/eObZ5+PxQw899M4nHn/LW95yMKyGUMexDEMSYG1Vh1BKWa/X
maxWq3EcgsQqkKTWmqFQLVAVSKMCKpBEBVQgiQqoJCaVqAdlGMexlKIC0WPrASXJSIAkQ1BJqg7DEEH
hHMchBNZ1XFFqrUmASlSqRR1jEmolLKSjDEJ1QJVK9mgmskYM5GkulFrTVJrLWw1Xq+d1FqTj00uju
PopDZqkn/5xx/PktTsMtR0P3BfffrLh4erYTgYDspAWa1WQyilrIbBpJTCa0pJItkAMPeQjEnIoIaah
GctgAKmGceRiVprTcl3GMklf/bif/vf/Nrf//v/1fHNmxnXpZQxAK6GkGSMsVaF0QzDwZ889/wn/49P
/vW/+teAJKwGN9ZjmYzjmaQ4ruOKopKYALVWoJRSawWSAGNMUgzJ0qbgWfCufVArGUISQK21AtFaeFV
1KOUYhxoSSqnkk5/85JNP/p8f/shH1sfH0masbMQunNREMwxDKauvfOUrv/Vbv/Whv/HvP/zww2NMqh
XIxEk0MMZSctUxZk1jEncICWotdYktVYyxng0qRN1HEe1TtQ6SfLj73wiS1Kzy1DT/cB95ctPXbh4e
TUMZcVAGSZAaZKUUpIASYA0EoIXcSETJVRcNImaxAmvqW4A2QBr5WD1j/7Rb3zoQx8arBZqrUMYx3G1
WtVak1SiHh8fHx4eqoCaRk3yqU/93uHR0Qf+4vtrrZk4YaIctVbAjQLVSoagllLGWEqptVJNwkrNAiS
ptQKVFENSFAi1llKSWFCBYqKU8k/+6Sf/pR/7sUcfeaTwdWLCMAwqo0YwgJoEUIEkWcc+8V9/5CP/2U
EZNSZYzNqaZEUZY17PSZJaK5DESRL/HEnGcawTtTZqrTXJOI5qrTXJj7/ziSxJzS5DTfcd98UvfuGey
5dLKavVahgok4MyLCZJKSUGAXIryIavSiEJRgVUQM1ELSYQX5WE1VCP10CG8l9+9GMf/9hHXY9JKimG
xER9/Cf+LS8/9YX1ej3G3/md3/ngBz+YBPi5n/u5/+0f/+NrV6+WUtiw1nDz5vrX/+E//E/+4/9oWJX
18QhYSFLMhpMCJpUMwJLGWmCsNUNhXf6Ho4MbDAdr3zJc+fmsbtb1wXpIcSMNoCyBNAAwitkAysHq7/
zyf/Df/YN/U0taLeZ7Di5e+vJHP/7Nb3zj0qVLq9Xq8PDw6OhgtTr88If/05s3byYBSinj0JZS10ef/
5PPff7z/+6/82/X0hbZSAKmkSowxiTFqJVsqEmptSbRMYlaazacjOPopE7GcUxSa1Vroz7+xLuzJDW7
DDXdD9wXv/iFey5fBoZhODgYymRfWQ1DYCgFqIXXZALkBEmqQBoVE82keOljLeZ7xrharT72sV/5+K9

87PjmTbWY16i11p/6wAd/7/c+tv6vV6tVklprkLLKhZ/8n//qr/4qri011iTA1atX/8f/6X/+D3/57z
gBkqjAGIvZKKU4GeMQKhmcuSfvXL1yQcfvllzIRnH+kv1JsVSR5NK2KgClahJinkNUEKxQCUBf/fv/
hef+MTfo8pqqLUCSf7J//5P3/++9/3ar/3aL/7ih5IMw5AE+JW/94mPfPjDP//zP//7n/m0CmRSyQsv
/Nnv/u7v/pW/8rMZHaMEs0E1CVCJJEyATNUmt0TGJkqTwtZqUOnEyjm0tNUmtdb1e29Ra1cefeHeWpGa
Xoab7gXvmmacLHBwcAAcHqYllGIYVBSilrFarJEWqYZIT1CSUES2txlpJCfFVdQBD9VW10pQhZGMov/
Ebv/FLv/TLx9evWVCTFLMBqEnKweql15+8KGHjm9eX6+Ph5QkFmosYQif/8JTX/zSv/j3fvZn11Y1k
1KKCtRaATUTNRuFoQZIUzcIkAZIUsz3jDGJmkIJxVRSzEYLg8Dn/ujzv/epT//tv/2LQ0hCYnJjPf72
b//2K6+8cjwZx/H4+Hi9Xt+4cQP44Ac/+IEP/0tYUwag1vriiy/+wR/8Xz/zM3+J6qsKmahJVCATNYm
aBFBrrTDUWnVMoiRZr9fAOI7qOI5qrVWtr5cEeOfj78qS10wy1HQ/cC9tFpVFYRgODw9LyTAMpZRhGE
opwDAM6kEZMpRiNlgNadQkQBIVSKJm4qSYSoozYzEqkEst5DVUk5RSMpT/5X/9ze+89NLF+lt/0/U4D
MMYqY6RoWBeozI5Pj7+6Ec/+tM//W/+zM/8pbIRSVGTVKIyUZOWUU2ztpZS1DTFjCUlFLNRCeAGwaiA
CiRRgSQqoCYBktRah2FQj460fvM3f+vg40Av/+v/i2oBk0qKqerWtdYhJAFevvLKr//6f/8Lv/ALjz7
61rJhKlGTAMWoLbxGzQRQk6hJ8VWjmhS1ToD1eg3UWtfrNTB0ktRax3F0kkR94l0/kSWp2Wwo6f7/8K
UvfffGtTo40CiLDAPDBF-itVqUUmLJKklJKvj+MRE2iFqMmqbVaqLxm9dQhjDEJkEQFVEcttQ7DcHx8f
0XKlVrrhQsXjo60GfrrwcFBknFcF9lQk1Ty/6ICaQ1CVBrBZK4QUpQSyLqCqkmUZMAOUeF1EzYqCyp
pYwxE8lQQ3JsHYbhc5/73Le+9a3r129+97vFPT4+vnjx4oULF4aB97znPQ888ACTUopaSlFrrUlKKUm
KswADJFGBWitQa4UkJPFVJKm1JlFrrWqdqHWijuNYa1XrRE0CPPGun8iS10wy1GwDyJLUvGE8/fSXh1
KAg4Nho0xWq1UpBVhRNsY4DAOTJICaphi1ku+hupFkjEmOLtyTM3XllZeSALXWUkqtdRiGi5fel0bKK
y+luXzP/Znzyne/DSS5fM/90VNXXnmp1lpKUZMAKpDECZBEzRxAzQRQc4shHNeRiQqoSULSa83EE2qt
6ji0ap2o6/U6iVonSdRSiro60Lh48fLb3/72dCegZhtAlqTmjeRLX/riUMpqtSqlHBwMQCnL40AgSSk
FKB0g1KImATJR83pqJmqTncmly/flTL3y3W+reb173/TmNC9/50Ugifqm+x7Mn0++/GeAeu+b3pwz9f
J3XkwC1Fp5TTVJJsqgJgHUNICaZBEzUQF1LyqJNGRSa01iQpDklprEie1VhWotaq1VnUcxzpRa61qn
eT/USmrCxcu/0iP/Li60ajZBpAlqXmD+aPP/+HBwcHh4WEpZbValZLVagWsVqtSSpJhGJgkKaV4gRI
oiaptQJJ1ETNxj33PpAz9Z2XXLABNQmQ5L77H0rz0re/lQnkvvvfknfeemFT067/6GcqZe+/S0134e
ahIkKqPlzJalqGkBNsiY6JgGS1FphUDNxAqi11iRqrTWtegt1HMdaq5Naa5qqLhpwePHxH38i3feBmm
0AWZKaN6TPf/4PDw6Hw9XRMAyr1QostwDKBFCSmkRJKqAmUfKqmkSpdQ0880aHc6Ze+NY3mSQ1Sa0Ve
PChR9K8+MI3kgKoDz70c0a8+MKfZvLgQw/nTL3wrW8mAZOoeVVJo2NeVdKoSYAkar4PNUkpRa21AknG
cQSSqEm8BTC0YxIn46TWmkSttapJ1FjW403K4cVL5fLFe97+wz+S7vtDzTaALEnNG9U4jp/7w392dHR
0cHCQ50DgoJQCDMOQZBgGoJSiMsLEtaICTVZArbUCSd76yGNpvh1ryVRSym11iSlFDUJkEsttQKllC
QqoD7y6NvS/0k3v57Jw299NH0++Y0/AdQkb33ksTTf+PrXmDgB1EcefVuaP/3m1zNRmdQJkyQqoAJJa
q2A+uhjP5TmG1//mgqogFpKcZJbqEmA3KLWCqiAmjLqEhVQkzhJoiZRa61J1Fqrk1qrWmtVa61Jaq2Z
uD7m0uEFLj7x7nelux3UbAPIktS84T355GcoXjy6BKxWqyTDMNCUUGAVUAEnQBo1k3f8hR9N88xXv5J
ETQOoQCZAehXILX74HT+S5rlnv5oE+KG3vyNznnv2q5moP/yOH0nz3LnfVQE1CaD+8Dt+JM2zzzydhE
mtNZNSSpJxHJmogJNSipN3/IUFTfPsm09nojJxkkQFMlGBWiuTwuQRgVUQE3CpNaqJLEBNQngnFprk
lqrTa3V1yQqCHR0dP3G+N6ffe+6040abQBZkpruFp/9/c9Q3SiLDMPApJSShEkmgBMgiVpKUd/9E/9q
mj/6/B8CaibqMAy11lJKJiqQRk1SSnn8iXeneoL6wC73z8XZnzhX/+R5kA73z8XWme+sIfAyqgZvL
Ox9+V5otP/fM0ak5QgUwANYn6+BPvTvPUF/5YTQKoSYBaayml1poT1CRqKUUFMnFSSqm1JgHUTJwAmT
hJU2tV6hAerXWal7F5Morxx/86Z9Ktz3UbAPIktR030et9Q8+85nvXruyXq+BYRiAJKUoJSiJgGSA
Oq/8YGfTv07n/qdTIDMAZIAahrgfe//qTRPfvbTapL3vu8vZs6Tn/10EkD9yfe+P82Tn/10EkAF1FLK
e/6196b5/Sc/A6iZqEAmKpBGzS1+8r3vT/PkZz+dBmgt1ExUQAUYUfP9qUmATNRMADWvByRRk5j4moz
jzVK9ds/Fb7/nfX813f83qOn2F5BGTQPkVGoaII2aCZA5auYAadRMgDRqGiCnUtMAadTsMiBLUrPLUN
PtLyCNmgbIqdQ0QBo1EyBz1MwB0qiZAGnUNEBOpaYB0qjZZUCWpGaXoabbX0AaNQ2QU6lpgDRqJkDmq
JkDpFEzAdKoaYCcSk0DpFGzy4AsSc0uQ023v4A0ahogp1LTAGnUnAckUdMAadScAKRR0wA5lZoGSKNm
lwFZkppdhppufwFp1DRATqWmAdKoQFIo6YB0qg5AUijpgFyKjUNKEbNLgOyJDW7DDXd/gLSqGmAnEp
NA6RRcwKQRk0DpFFzApBGTQPkVGoaII2aXQZkSWp2GWq6/QWkUdMAOZWaBkij5gQgjZoGSKPmBCCNmG
bIqdQ0QBo1uwzIktTsMtR0+wtIo6YBcio1DZBGzQRio+Z2gDRqJkAaNQ2QU6lpgDRqdhmQJanZZajZB
pAlqen0DpBGTQPkVGoaII2aCZBGze0AadRMgDRqGiCnUtMAadR0+ws12wCyJDXd2QHSqGmAnEpNA6RR

MwHSqLkdII2aCZBGTPKvGoaII2abn+hZhtAlqSmOztAGjUnkFOpaYA0aiZAGjW3A6RRMwHSqGmAnEp
NA6RR0+0v1GwDyJLUdGcHSPmrgBp1EyANGrmaJmjZgKkUXNXgDRquv2Fmm0AWZKa7uwAadTcFSCNm
mQRs0cIHPUTIA0au4KkeZNT79Qsw0gS1LTnR0gjZq7AqRRMwHSqJkDZI6aCZBGzV0B0qjp9hdqtgFkS
Wq6sw0kUXNXgDRqJkAaNx0AzFEzAdKouStAGjXd/kLNN0AsSU13doCcKTUTII2aBkijpgHSqJkA0VNq
uv2Fmm5/ATlTaiZAGjUnkeZNA6RRMwFyptTsmiBLurPLUNPtLyBnSs0ESK0mAdKoaYA0aiZazpSaXQZ
kSWp2GWq6/QXkTKmZAGnUNEAaNQ2QRs0EyJlSs8uALEnNLkNN120PyN1S080BsiQ1uww1Xbc9IHdLTT
cHyJLU7DLUdN32gNwtNd0cIEtSs8tQ0+0vICeouQWQiZo5QE5Q0wCZo+Z2gJxKTQPkBDX7AsiS10wy1
HT7C8gcNRMgjZo5QE5Q0wCZo+ZUQG5HTQPkBDX7AsiS10wy1GwDyJLUdGcHyBw1EyCNmjLATlDTAJmj
5lRABkdNA+QENd0bA2q2AWRJArqzA2S0mgmQRs0cICeoaYDMUXMqIlejpgFygprujQE12wCyJDXd2QH
SqLljQBo1DZAT1DRAGjV3DEijpgHSqDkBSK0m21+o2QaQJanpzg6QRs0dA9KoaYCCoKYB0qi5Y0AaNQ
2QRs0JQBo13f5CzTaALElNd3aANGruGJBGTQPkBDUNKebNHQPSqGmANGp0ANKo6fYXarYBZElqurMDp
FFzx4A0ahogJ6hpgDRq7hiQRk0DpFFzApBGTbe/ULMNIETs050dII2aBsgJahogp1LTAJmjpgFyx9Sc
CkijpntjQM02gCxJTXd2gDRqGiAnqGmAnEpNA2S0mgbiHVNzKiCnmu6NATXd/gLSqGmAnKCMAXIqNQ2
QOWoaIHdMzamANGr2BZAlqdlLqOn2F5BGTQPkBDUNKFOpaYDMUDMAuWNqTgWkUbMvgCxJzS5DTbe/gM
xRMwHSqLkdIKdS0wBp1JwApFHTAGnUNEAmaVYRkCWp2Wwo6fYXkdLqJkAaNbcD5FRqGiCnMh0ANGoaI
I2aBshEzT4CsiQ1uww13f4CMkfNBEij5naAnEpNA6RRcwQQRk0DpFHTAJmo2UdAlqRml6Gm219A5qiZ
AGnU3A6QU6lpgDRqTgDSqGmANGoaIBM1+wjIktTsmTR0bwxAGjV3DMgdUzMHSKNmAqRR0wCZo2YCZI6
aXQZkSWp2GWq6NwYgjZo7BuS0qZkDpFEzAdKoaYDMUTMBmkfNLgOyJDW7DDXbALIkNd0ygdRq7hiQ06
ZmDpBGzQRIO6YBMkfNBmgcNd3+Qs02gCxJTbcMIBM12wByx9TMATJR0wBp1DRA5qiZAJmjpttfqNkGk
CWp6XYBkEbNHCCNMh0ANGoaII2aBshETQOk3t2PMgAABxxJREFUUDPtL9RsA8iS1HS7AEijZg6QRs0J
QBo1DZBGTQNKoqYB0qjp9hdqtgFkSWq6XQCKUTMHSPmBCCNmGZIo6YBMlHTAGnUdPsLNdsAsiq13dk
BcqbUTIDMUXM7Q06KmgmQOWq6/YWabQBZkpRU7AA5U2omQ0aouR0gd0XNBmgcNd3+Qs02gCxJTXd2gJ
wpNRMgc9TcDpC7omYCZI6abn+hpttfQM6UmgmQOWpuB8hdUTMBmkfNLgOyJDW7DDXd/gLSqLkRQBo1J
wC5K2rmAGnUNEAmaVYRkCWp2Wwo6fYXkEbNXQHSqDkByF1RMwdIo6YBMlGzj4AsSc0uQ023v4A0au4K
kEbNCUDuipo5QBo1DZCJmn0EZEldhLquv0FpFFzV4A0ak4AcLfUzAHSqGmATNTsiYBLurPLUNPtLyC
NmgbIqdQ0QBo1JwBp1DRAGjUNKBPU3DEgjZp9AWRJAnYZarr9BaRR0wA5lZoGSKPmBCCNmGZIo6YBco
Ka0wakUbMvgCxJzS5DTbe/gDRqGiCnUtMAadScAKRR0wBp1DRATlBzx4A0avYFkCWp2Wwo2QaQJanpz
g6QRk0D5FRqGiCnMh0ANGoaII2aBsgJau4YkeZN98aAmm0AWZKa7uwAadQ0QE6lpgHSqJkA2YaaUwGZ
o6YBcio13f5CzTaALElNd3aANGoaIKdS0wBp1EyAbEPNqYDMUDMAOZWabn+hZhtAlqSmOztAGjUnkFO
paYA0aiZatqHmVEDmqGmAnEpNt79Qsw0gS1LTnR0gjZoGyKnUNEAaNQ2Q06bmVEDmqGmAnEpNt79Qsw
0gS1LTnR0gjZoGyKnUNEAaNScAadTMAdKouStAJmoaII2abn+hZhtAlqSmOztAGjUnkFOpaYA0ak4A0
qiZA6RRc1eATNQ0QBo13f5CzTaALElNd3aANGoaIKdS0wBp1JwApFEzB0ij5q4AmahpgDRquv2Fmm5/
AWnU3BUGjZoGyAlqbgfICWrmADlBzT4CsiQ1uww13f4C0qi5K0AaNRMgc9TcDpAT1MwBcoKaFQRkSWp
2GWq6/QWkUXNXgDRqJkDmqLkdICEomQPkBDX7CMiS10wy1HT7C0ij5q4AadRMgMxRczATlAzB8gJav
YRkCWp2Wwo6fYXkd0LZgJkjpGyBw1JwC5K2r2BZAlqdlLqOn2F5AzpWYCZI6aBsgcNScAuStq9gWQJ
anZZajp9heQM6VmAmS0mgbiHDUnALkravYFkCWp2Wwo6fYXkd0LZgJkjpGyBw1JwC5K2r2BZAlqdlL
qNkGkCWp6bqum40abQBZkpqu67o5qNkGkCWp6bqum40abQBZkpqu67o5qNkGkCWp6bqum40abQBZkpq
u67o5qNkGkCWp6bqum40abQBZkpqu67o5q0m67nwAsiQ1uww1XdedD0CWpGaXoabruvMByJLU7DLUDF
13PgBZkppdhphu684HIEtSs8tQ03Xd+QBkSWp2GWq6rjsfgCxJzS5DTdd15w0QJanZZajZBpAlqem6r
puDmm0AWZKaruu60ajZBpAlqem6rpuDmm0AWZKaruu60ajZBpAlqem6rpuDmm0AWZKaruu60ajZBpAl
qem6rpuDmm0AWZKaruu60ajpuu58ALiKnbsMNV3XnQ9AlqRml6Gm67rzAcis10wy1HRddz4AWZKaXYa
aruv0ByBLurPLUNN13fkAZEldhLquq47H4AsSc0uQ03XdecDkCWp2Wwo2QaQJanpuq6bg5ptAFmSmq
7rujmo2QaQJanpuq6bg5ptAFmSmq7rujmo2QaQJanpuq6bg5ptAFmSmq7rujmo2QaQJanpuq6bg5ptA
FmSmq7rujmo6brufACyJDW7DDVd150PQJakZpehpuu68wHIktTsmTR0XXc+AFmSm12Gmq7rzgCGS1Kz
y1DTdd35AGRJanYZarquOx+ALEnNLkNN13XnA5AlqdlLqNkGkCWp6bqum40abQBZkpqu67o5qNkGkCW
p6bqum40abQBZkpqu67o5qNkGkCWp6bqum40abQBZkpqu67o5qNkGkCWp6bqum40abQBZkpqu67o5q0
m67nwAsiQ1uww1XdedD0CWpGaXoabruvMByJLU7DLUDF13PgBZkppdhphu684HIEtSs8tQ03Xd+QBkS

```
Wp2GWq6rjsfgCxJzS5DTdd15wOQJanZZajZBpAlqem6rpuDmm0AWZKaru60ajZBpAlqem6rpuDmm0A
WZKaru60ajZBpAlqem6rpuDmm0AWZKaru60ajZBpAlqem6rpuDmm0AWZKaru60ajp58ALIkNbs
MNV3XnQ9AlqRm16Gm67rzAcis10wy1HRddz4AWZKaXYaaruv0ByBLUrPLUNN13fkAZE1qdh1quq47H4
AsSc0uQ03XdecDkCWp2WX/N3y+mOLHwCY+AAAAAE1FTkSuQmCC\",\\n\\t\\\"errorCode\\\" :
0,\\n\\t\\\"errorInfo\\\" : \\\"No error!\\\"\\n\\n\",
```

```
7         "result": 0
8     }
9 }
10
11
12
13 // 创建打印实例,此实例只需创建一次
14 this.nMPrintSocket = new NMPrintSocket(socketData);
15 // 调用流程
16 async generateImagePreviewImage(){
17     const generateImagePreviewImageParam = {
18         "displayScale":8
19     }
20
21     const res = await
this.nMPrintSocket.generateImagePreviewImage(generateImagePreviewImageParam['di
splayScale']);
22     //预览图生成失败，退出流程
23     if (res.resultAck.result !== 0) {
24         return;
25     }
26
27     //解析处理数据
28     const obj = JSON.parse(info);
29     const data = obj.ImageData;
30
31 }
```

四、打印接口说明

4.1 设置打印回调

代码块

```
1 export default class NMPrintSocket {
2     // 添加打印监听方法
3     addPrintListener(callback)
4 }
```

代码块

```

1  let printListener = null;
2
3  printListener = this.nMPrintSocket.addPrintListener(async (msg) => {
4      const resultAck = msg?.resultAck;
5
6      if (resultAck?.errorCode === 0 && resultAck?.info === "commitJob ok!") {
7          await strategyFactory.handleCommitSuccess();
8      }
9      //已接入历史版本客户仍可以使用printQuantity和onPrintPageCompleted字段获取打印进度
10
11     if (resultAck?.printCopies !== null && resultAck?.printPages !== null) {
12         strategyFactory.handleProgressUpdate(resultAck);
13     }
14
15     if (resultAck?.printCopies === printQuantity &&
16         resultAck?.printPages === list.length) {
17         await strategyFactory.handleCompletion();
18     }
19
20     if (resultAck?.errorCode !== 0) {
21         strategyFactory.handleError(msg);
22     }
23 });

```

4.2 移除打印回调

代码块

```

1  export default class NMPrintSocket {
2      /**
3       * 移除指定的打印监听回调函数
4       * @param {Function} callback - 需要移除的回调函数
5       * @returns {void}
6       */
7      removePrintListener(callback)
8  }

```

代码块

```

1  const cleanupListener = () => {
2      if (printListener) {
3          this.nMPrintSocket.removePrintListener(printListener);
4          printListener = null;
5      }
6  };

```

4.3 开始打印

代码块

```
1  export default class NMPrintSocket {
2      /**
3       * 开始一个打印任务。
4       *
5       * @param {number} printDensity - 打印浓度，根据不同打印机型号取值范围不同，具体
      如下：
6       *
7       *                                     - B3S、B203、B1、K3、K3W、M2：取值范围
      1~5，默认为 3。
7       *                                     - B50、B11、B50W、B32、Z401：取值范围
      1~15，默认为 8。
8       * @param {number} printLabelType - 纸张类型，可选值：
9       *                                     1：间隙纸
10      *                                     2：黑标纸
11      *                                     3：连续纸
12      *                                     4：定孔纸
13      *                                     5：透明纸
14      *                                     6：标牌
15      *                                     10：黑标间隙纸
16      * @param {number} printMode - 打印模式，可选值：
17      *                                     1：热敏
18      *                                     2：热转印
19      *                                     注意，不同打印机型号支持的打印模式有限制，具体如下：
20      *                                     - D11、D101、D110、H10、B16、B18、B3S、B203、
      B1、K3、K3W、B11 仅支持热敏。
21      *                                     - B50、B50W、B32、Z401、M2 仅支持热转印。
22      * @param {number} count - 总打印份数，表示所有页面的打印份数之和。
23      *                                     例如，如果你有3页需要打印，第一页打印3份，第二页打
      印2份，第三页打印5份，那么count的值应为10（3+2+5）。
24      * @return {Promise} - 返回一个 Promise，解析为开始打印任务的结果
25      * @example
26      * //返回数据示例
27      * {
28      *     "apiName": "startJob",
29      *     "resultAck": {
30      *         "errorCode": 0,
31      *         "info": "startJob ok!",
32      *         "result": 0
33      *     }
34      * }
35      * @description 返回结果中的 errorCode 含义如下：
36      *                                     - 0：成功
37      *                                     - -1：失败，info 表示原因
```



```

38      *          - -2: 打印机忙碌, info 表示原因
39      *          - -3: 打印机接收到不支持的参数, 主要是浓度、纸张类型、打印模式,
      info 表示具体原因
40      */
41      startJob(printDensity, printLabelType, printMode, count) {
42          // 方法实现将在这里
43      }
44  }

```

代码块

```

1  //返回数据示例
2  {
3      "apiName": "startJob",
4      "resultAck": {
5          "errorCode": 0,
6          "info": "startJob ok!",
7          "result": 0
8      }
9  }
10 // 创建打印实例,此实例只需创建一次
11 this.nMPrintSocket = new NMPrintSocket(socketData);
12 //总打印页数
13 const printDataLength = 5;
14 // 调用流程
15 async startJob() {
16     const jsonObj = {
17         printerImageProcessingInfo: {
18             printQuantity: 1,
19         },
20     };
21     const density = 3;
22     const label_type = 1;
23     const print_mode = 1;
24     const printQuantity = jsonObj.printerImageProcessingInfo.printQuantity;
25     try {
26         const startRes = await this.nMPrintSocket.startJob(
27             density,
28             label_type,
29             print_mode,
30             printDataLength*printQuantity
31         );
32         if (startRes.resultAck.result == 0) {
33             // 提交打印任务
34             await this.printTag(list, 0);
35         }

```

```

36     } catch (err) {
37         console.error(err);
38     }
39 }

```

4.4 提交打印任务

代码块

```

1  export default class NMPrintSocket {
2      /**
3       * 提交打印任务，并执行回调函数。
4       *
5       * @param {string} [printData=null] - 打印数据的 JSON 字符串。
6       * @param {string} printerImageProcessingInfo - 打印机图像处理信息的 JSON 字符串，包含打印份数信息，格式如下：
7       * {
8       *   "printerImageProcessingInfo": {
9       *     "printQuantity": 1 // 用于指定当前页的打印份数。例如，如果需要打印3页，第一页
          打印3份，第二页打印2份，第三页打印5份，则在3次提交数据时，printerImageProcessingInfo
          中的 "printQuantity" 值分别应为 3, 2, 5。
10      *   }
11      * }
12      *
13      * @return {Promise} 返回一个 Promise，解析为提交打印任务返回信息
14      *
15      * @description
16      * 需要先开启打印任务，完成绘制后再提交打印任务
17      */
18      commitJob(printData, printerImageProcessingInfo)
19  }

```

代码块

```

1  //数据提交成功返回数据示例
2  {
3      "apiName": "commitJob",
4      "resultAck": {
5          "errorCode": 0,
6          "info": "commitJob ok!",
7          "result": 0
8      }
9  }
10
11 //打印进度返回示例1：此回调的含义为第一页第一份打印完成
12 {

```



```

13         "apiName": "commitJob",
14         "resultAck": {
15             "errorCode": 0,
16             "info": "",
17             "onPrintEPCCodeCompleted": "",
18             "onPrintPageCompleted": 1, //打印完成份数回调
19             "onPrintPageLengthCompleted": "38.00",
20             "printQuantity": 1 //打印完成页数回调
21         }
22     }
23
24     //打印进度返回示例1: 此回调的含义为第一页第二份打印完成
25     {
26         "apiName": "commitJob",
27         "resultAck": {
28             "errorCode": 0,
29             "info": "",
30             "onPrintEPCCodeCompleted": "",
31             "onPrintPageCompleted": 2, //打印完成份数回调
32             "onPrintPageLengthCompleted": "38.00",
33             "printQuantity": 1 //打印完成页数回调
34         }
35     }
36
37     //打印进度返回示例1: 此回调的含义为第二页第一份打印完成
38     {
39         "apiName": "commitJob",
40         "resultAck": {
41             "errorCode": 0,
42             "info": "",
43             "onPrintEPCCodeCompleted": "",
44             "onPrintPageCompleted": 1, //打印完成份数回调
45             "onPrintPageLengthCompleted": "38.00",
46             "printQuantity": 2 //打印完成页数回调
47         }
48     }
49
50
51     // 创建打印实例,此实例只需创建一次
52     this.nMPrintSocket = new NMPrintSocket(socketData);
53     //打印数据长度
54     const printDataLength = 5;
55     // 调用流程
56     async commitJob() {
57         const jsonObj = {
58             printerImageProcessingInfo: {
59                 printQuantity: 1,

```

```

60     },
61   };
62
63   try {
64     this.nMPrintSocket.commitJob(null, JSON.stringify(this.jsonObj));
65   } catch (err) {
66     console.error(err);
67   }
68
69 }

```

4.5 结束打印任务

代码块

```

1  export default class NMPrintSocket {
2    /**
3     * 结束打印任务
4     *
5     * @return {Promise} 返回一个 Promise, 解析为结束打印任务的结果
6     *
7     * @description
8     * 收到最后一页最后一份打印页面后调用该函数结束打印任务
9     */
10   endJob()
11 }

```

代码块

```

1  //返回数据示例
2  {
3    "apiName": "endJob",
4    "resultAck": {
5      "errorCode": 0,
6      "info": "endJob ok!",
7      "result": 0
8    }
9  }
10 // 创建打印实例, 此实例只需创建一次
11 this.nMPrintSocket = new NMPrintSocket(socketData);
12 // 调用流程
13 async endJob() {
14   try {
15     const endRes = await this.nMPrintSocket.endJob();
16     if (endRes.resultAck.errorCode == 0) {
17       console.log("结束打印完成");

```

```

18     }
19   } catch (err) {
20     console.error(err);
21   }
22 }

```

4.6 取消打印任务

代码块

```

1  export default class NMPrintSocket {
2    /**
3     * 取消当前的打印任务，并执行回调函数。
4     *
5     * @return {Promise} 返回一个 Promise，解析为取消打印任务的结果
6     */
7    cancelJob()
8  }

```

代码块

```

1  // 创建打印实例,此实例只需创建一次
2  this.nMPrintSocket = new NMPrintSocket(socketData);
3  // 调用流程
4  async cancelJob() {
5    try {
6      const cancelJobRes = await this.nMPrintSocket.cancelJob();
7      if (cancelJobRes.resultAck.errorCode == 0) {
8        console.log("取消打印成功");
9      }
10   } catch (err) {
11     console.error(err);
12   }
13 }

```

五、回调说明

代码块

```

1
2  /**
3   * {
4   *   "apiName": string, // 调用的 API 名称
5   *   "resultAck": {
6   *     "errorCode": number, // 错误代码, 0 表示成功, 其他值表示错误

```

```

7      *      "info": string, // 信息字符串, 描述操作结果
8      *      "result": number // 结果代码, 通常与 errorCode 一致
9      *      }
10     *      }
11     */
12     {
13         "apiName": "commitJob",
14         "resultAck": {
15             "errorCode": 0,
16             "info": "commitJob ok!",
17             "result": 0
18         }
19     }

```

六、错误码相关说明

6.1 错误码说明描述

代码块

```

1      * 0-无错误
2      //打印机返回部分
3      * 1-盒子打开
4      * 2-缺纸
5      * 3-电量不足
6      * 4-电池异常
7      * 5-手动停止
8      * 6-数据错误
9      * 7-温度过高
10     * 8-走纸异常
11     * 9-正在打印
12     * 10-未检测到打印头
13     * 11-环境温度过低
14     * 12-打印头松动
15     * 13-未检测到碳带
16     * 14-不匹配的耗材
17     * 15-用完的碳带
18     * 16-不支持的纸张类型
19     * 17-设置纸张类型失败
20     * 18-设置打印模式失败
21     * 19-设置浓度失败
22     * 20-写入rfid失败
23     * 21-边距参数错误
24     * 22-超时错误
25     * 23-断开连接
26     * 24-画板参数设置错误

```

```
27 * 25-旋转角度参数错误
28 * 26-json参数错误
29 * 27-出纸异常（关闭上盖检测）
30 * 28-检查纸张类型
31 * 29-碳带与打印模式不匹配
32 * 30-设置浓度不支持
33 * 31-不支持的打印模式
34 * 32-标签材质设置异常，请重新设置
35 * 33-不支持该标签材质，请更换或重新设置
36 * 34-不支持RFID写入
37 * 50-非法标签
38 * 51-非法碳带和标签
39
40 //内部使用
41 //E_UNKNOW_ERROR = 255,
```